Introduction to Large Language Models

Lecture 2: Language Modelling, GPT, Decoding Strategies

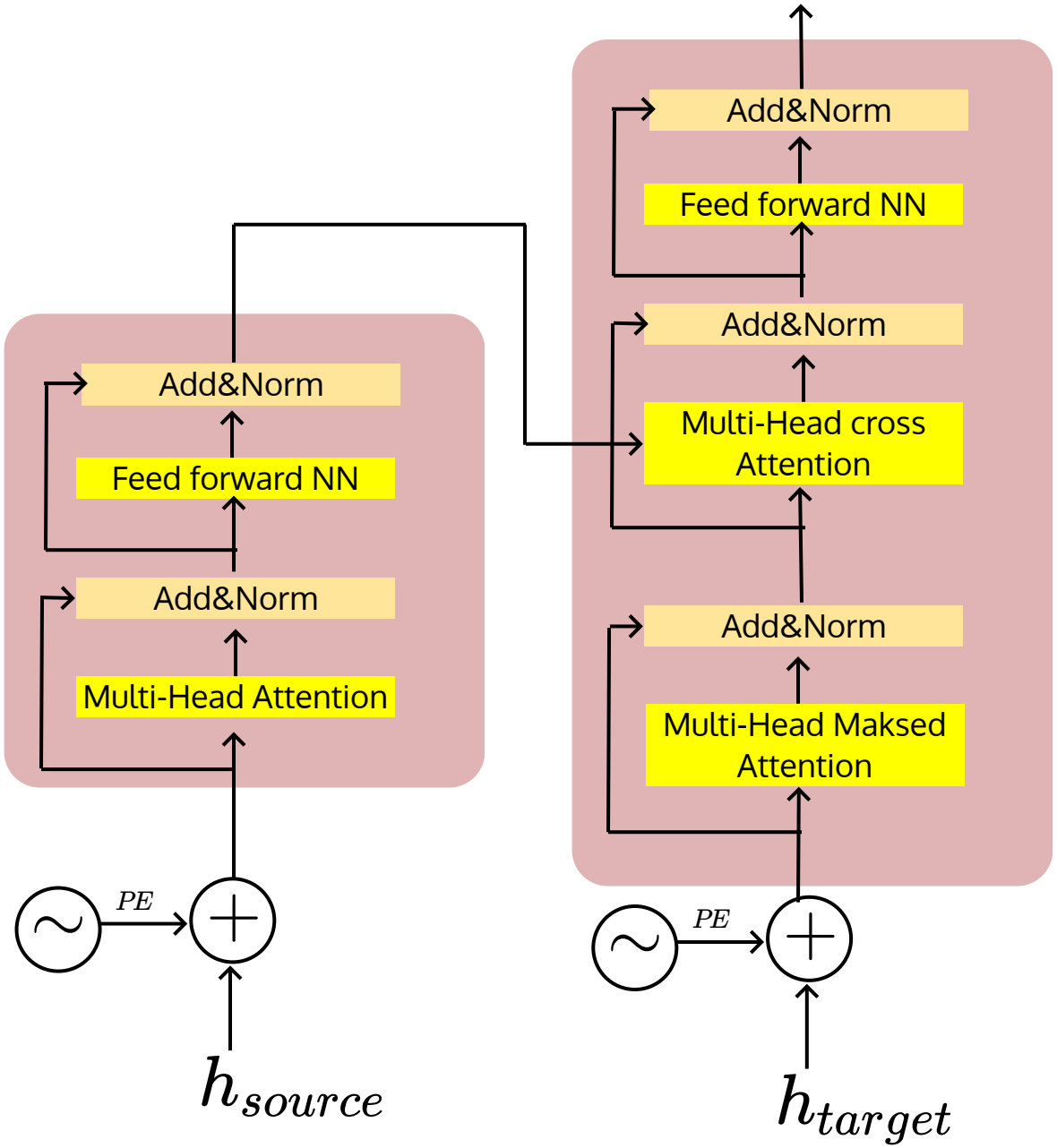
Mitesh M. Khapra





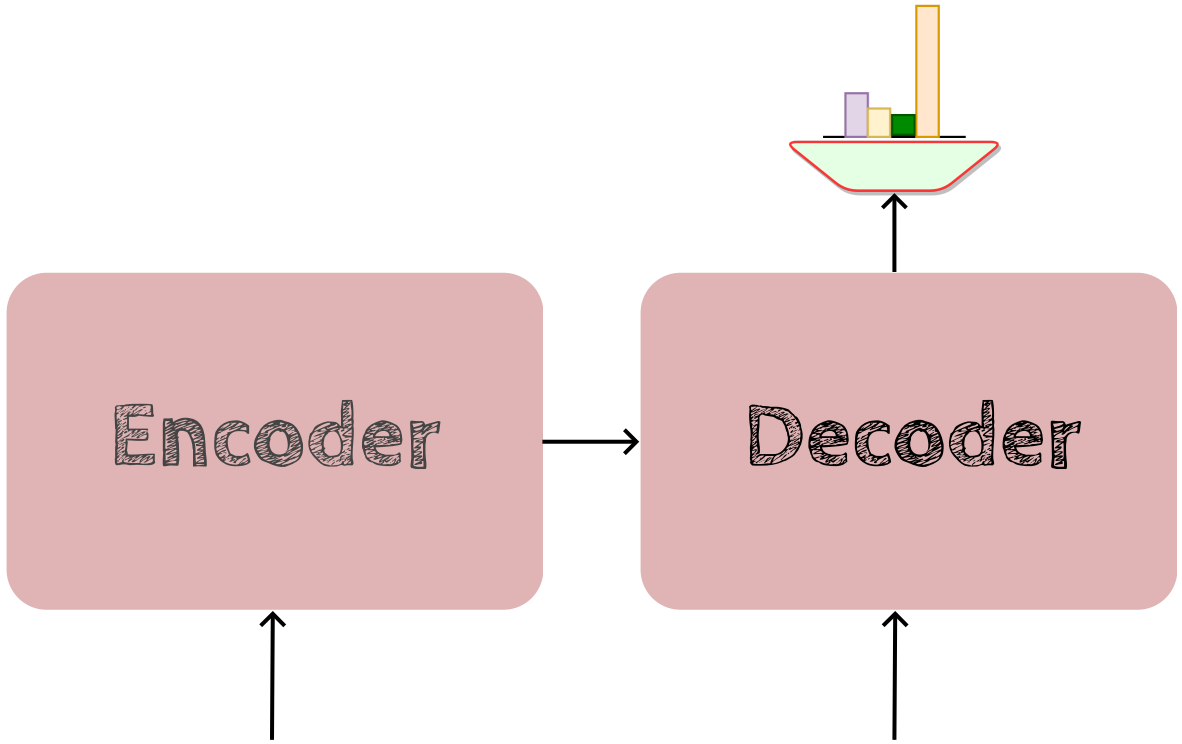
AI4Bharat, Department of Computer Science and Engineering, IIT Madras

In the previous lecture, we learned about the components of the transformer architecture in the context of machine translation.



In the previous lecture, we learned about the components of the transformer architecture in the context of machine translation.

At a high-level, we can just think of it as an encoder-decoder block



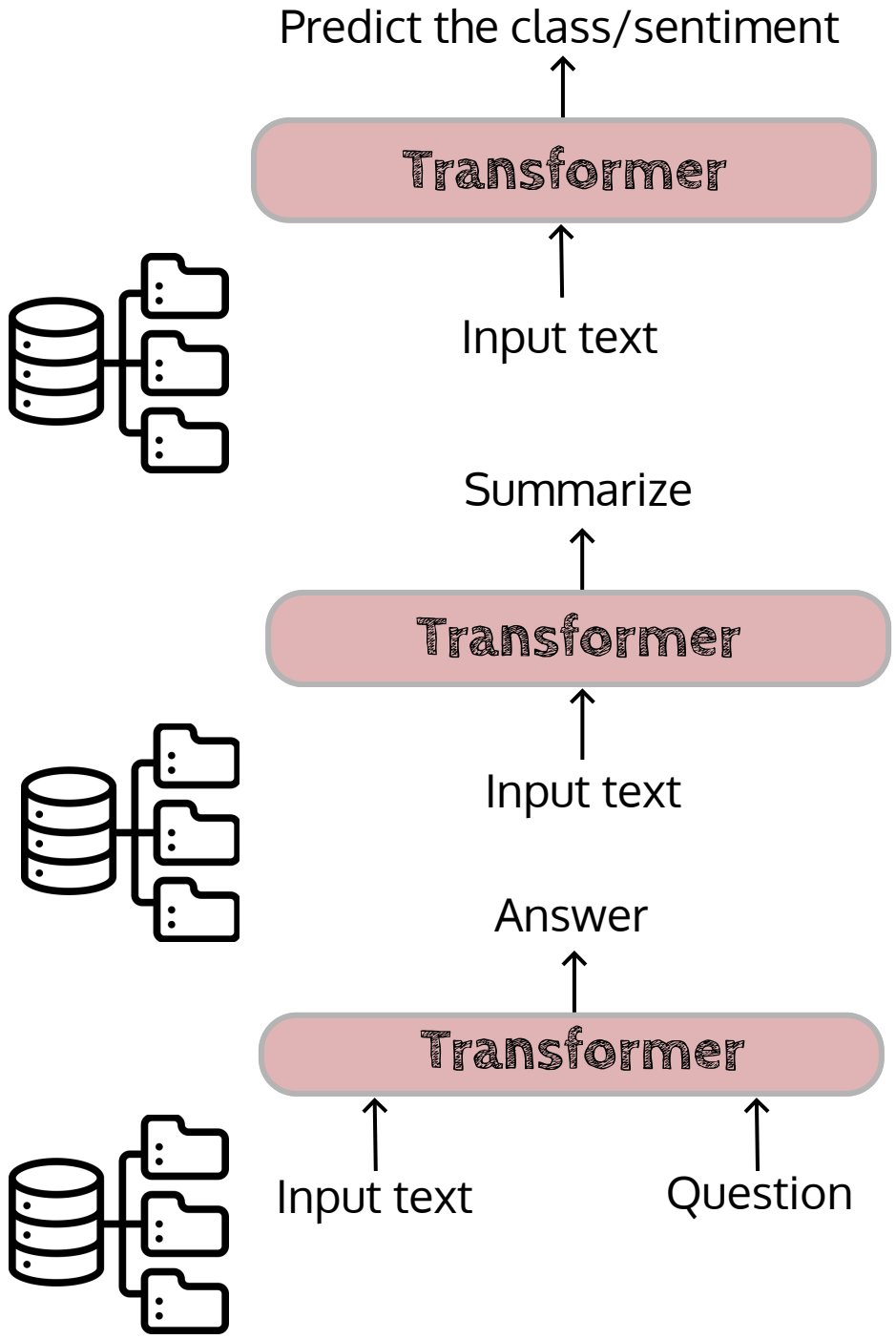
In the previous lectures, we learned the components of transformer architecture in the context of machine translation.

What if we want to use the transformer architecture for other NLP tasks?

We need to train a separate model for each task using dataset specific to the task

If we train the architecture from scratch (that is, by randomly initializing the parameters) for each task, it takes a long time for convergence

Often, we may not have enough labelled samples for many NLP tasks



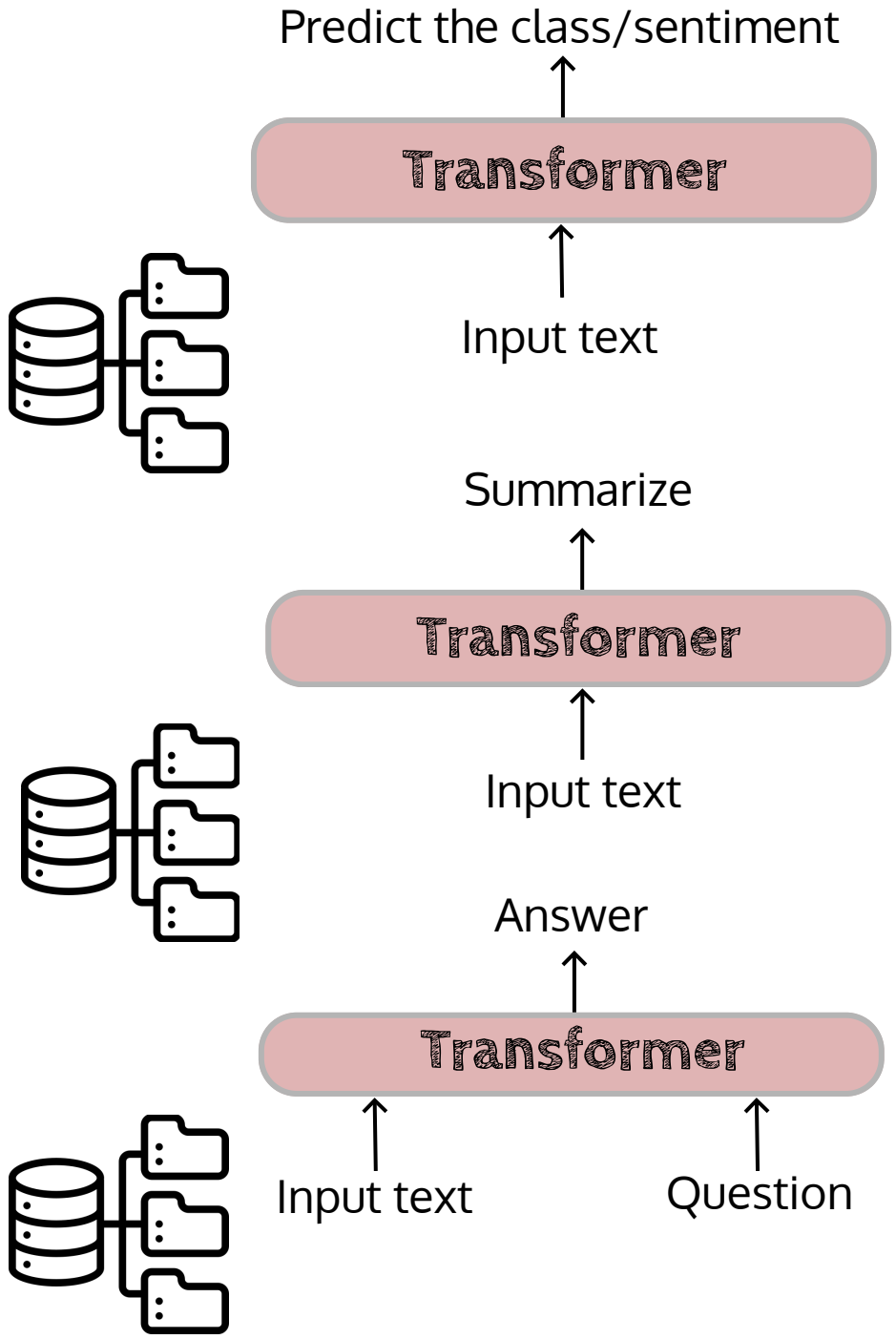
Moreover, preparing labelled data is laborious and costly

On the other hand,

We have a large amount of unlabelled text easily available on the internet



Can we make use of such unlabelled data to train a model?



Moreover, preparing labelled data is laborious and costly

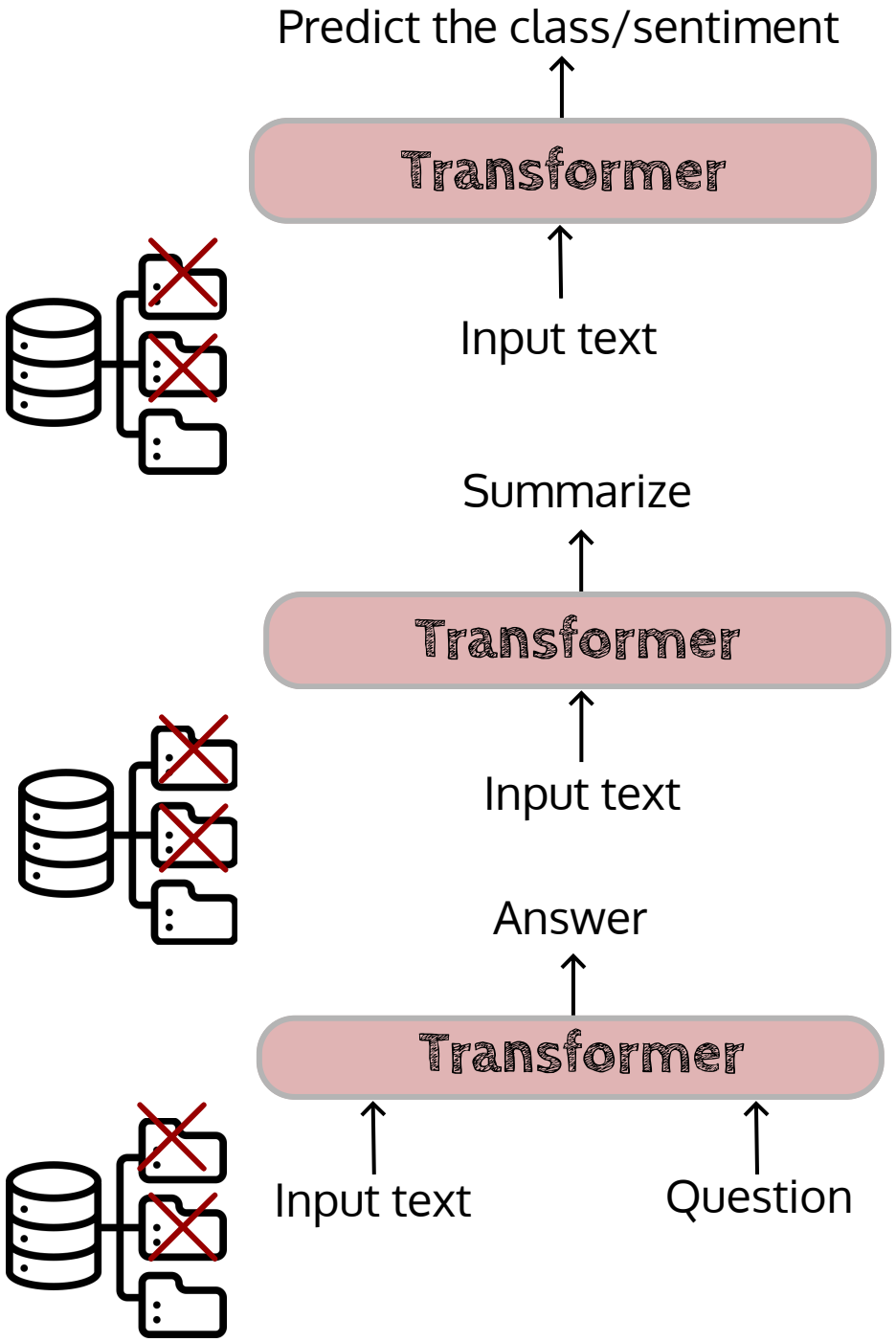
On the other hand,

We have a huge unlabelled text data on the internet

Can we make use of these unlabelled data to train a model?

What will be the training objective in that case?

Would that be helpful in adapting the model to downstream tasks with minimal fine-tuning (with zero or a few samples)?



Nitesh Gupta

Module 2.1 : Language Modelling

Mitesh M. Khapra



AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Motivation

Assume that we ask questions to a lay person based on a statement or some excerpt

" Wow, India has now reached the moon"

An excerpt from business today "What sets this mission apart is the pivotal role of artificial intelligence (AI) in guiding the spacecraft during its critical descent to the moon's surface."

He likes to stay

He likes to stray

He likes to sway

Is this sentence expressing a positive or a negative sentiment?

Did the lander use AI for soft landing on the moon?

Are these meaningful sentences?

The person will most likely answer all the questions, even though he/she may not be explicitly trained on any of these tasks. How?

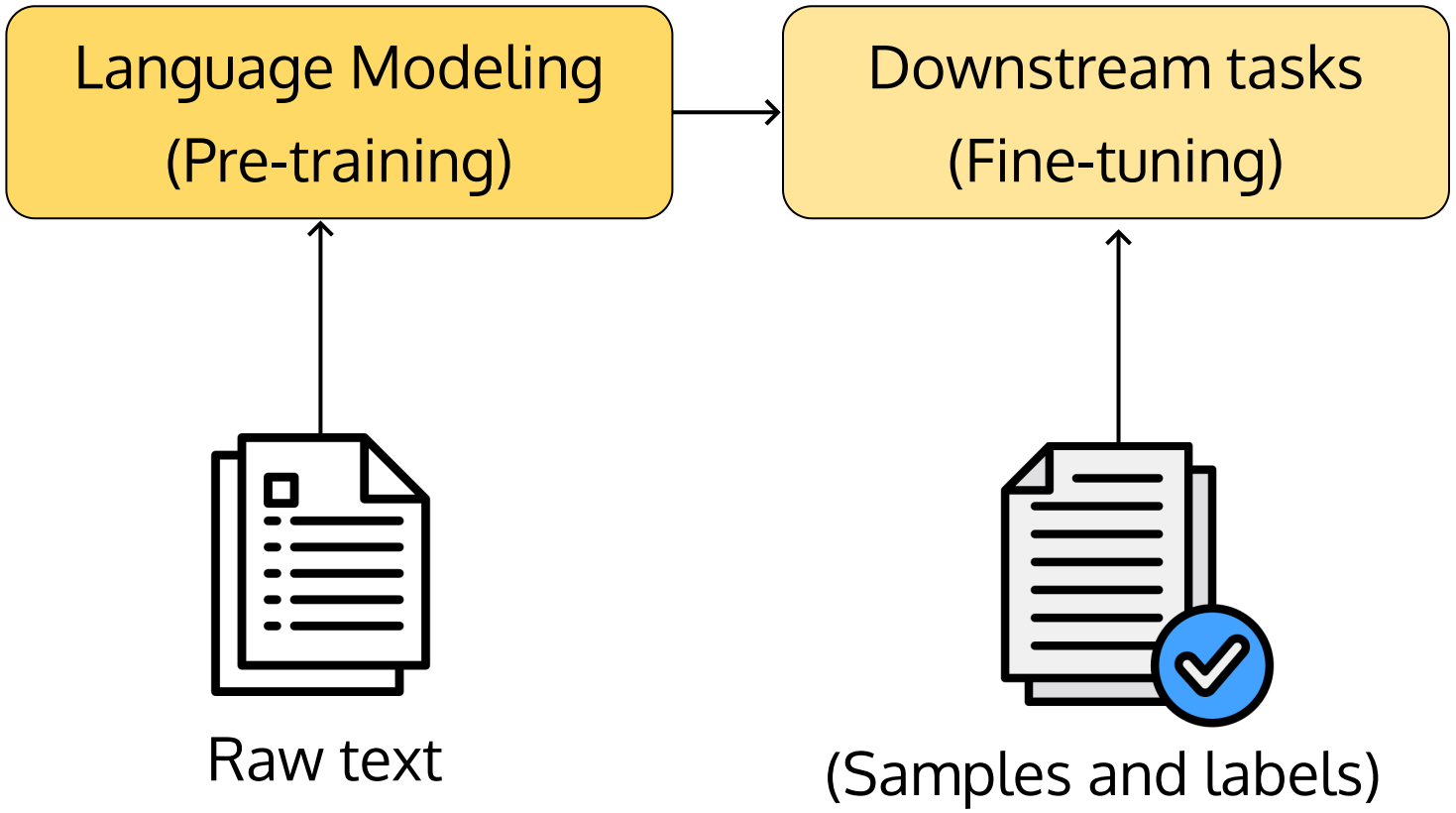
We develop a strong understanding of language through various language based interactions(listening/reading) over our life time without any explicit supervision

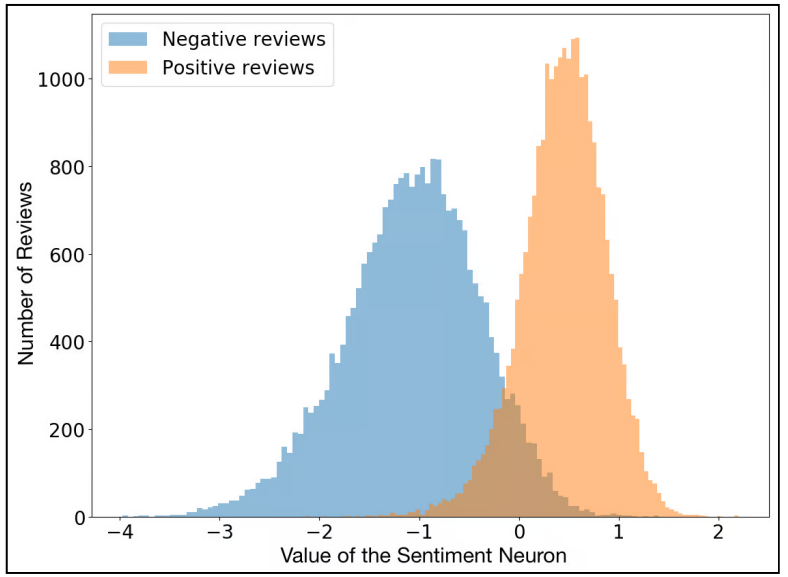


Idea

Can a model develop basic understanding of language by getting exposure to a large amount of raw text? [pre-training]

More importantly, after getting exposed to such raw data can it learn to perform well on downstream tasks with minimum supervision? [Supervised Fine-tuning]





With this representation a **linear model** classifies reviews with 91.8% accuracy beating the SOTA (Ref)

...matches the performance of previous supervised systems using **30-100x fewer labeled** examples (Ref)

Language modelling

Let $\mathcal{V}$ be a vocabulary of language (i.e., collection of all unique words in the language)

We can think of a sentence as a sequence $X_1, X_2, \cdots, X_n$, where $X_i \in \mathcal{V}$

For example, if $\mathcal{V} = \{an, apple, ate, I\}$, some possible sentences (not necessarily grammatically correct) are

a. An apple ate I

b. I ate an apple

c. I ate apple

d. an apple

e. ....

Intuitively, some of these sentences are more probable than others.

What do we mean by that?

Intuitively, we mean that give a very very large corpus, we expect some of these sentences to appear more frequently than others (hence, more probable)

We are now looking for a function which takes a sequence as input and assigns a probability to each sequence

$$f : (X_1, X_2, \cdots X_n) \rightarrow [0, 1]$$

Such a function is called a language model.

Language modelling

$$P(x_1, x_2, \cdots, x_T) = P(x_1)P(x_2|x_1)P(x_3|x_2, x_1)\cdots P(x_T|x_{T-1}, \cdots, x_1)$$

$$= \prod_{i=1}^{T} P(x_i|x_1, \cdots, x_{i-1})$$

If we naively assume that the words in a sequence are independent of each other then

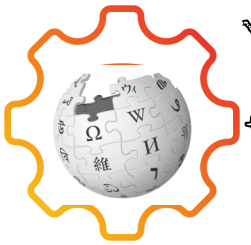
$$\boxed{P(x_1, x_2, \cdots, x_T) = \prod_{i=1}^{T} P(x_i)}$$



How do we enable a model to understand language?



Simple Idea: Teach it the task of predicting the next token in a sequence..



You have tons of sequences available on the web which you can use as training data

Chandrayaan-3

Article Talk

From Wikipedia, the free encyclopedia

Chandrayaan-3 (/ ˌtʃʌndrəˈjɑːn/ *CHUN-drə-YAHN*) is the third mission in the Chandrayaan programme, a series of lunar-exploration missions [redacted] by the Indian Space Research Organisation (ISRO).[7] Launched on 14 July 2023, the mission consists of a l[redacted] named *Vikram* and a lunar rover named *Pragyan*, similar to those launched aboard Chandrayaan-2 in 2019.

Chandrayaan-3 was [redacted] from Satish Dhawan Space Centre on 14 July 2023. The spacecraft [redacted] lunar orbit on 5 August, and the lander [redacted] down near the Lunar south pole[8] on 23 August at 18:03 IST (12:33 UTC), making India the fourth country to [redacted] land on the Moon, and the first to do so near the lunar south [redacted] On 3 September the lander hopped and repositioned itself 30–40 cm (12–16 in) from its landing site.[13] [redacted] the completion of its mission objectives, it was hoped that the lander and rover would [redacted] for extra tasks, on 22 September 2023,

BECAUSE he is the [redacted] in conceiving and implementing India's digital building blocks such as UPI, Aadhaar, eKYC and FASTag. [redacted] also helped the government develop the IT infrastructure to [redacted] GST and the Ayushman Bharat Yojana. The cerebral [redacted] is the government's go-to person for tech intervention, irrespective of the party in power

BECAUSE he was [redacted] in the launch of the Beckn protocol, now being used in sectors such as e-commerce, mobility and health. He was [redacted] the government-backed Open Network for Digital [redacted] aimed at helping small Indian retailers fight back against e-com giants such as [redacted] and Flipkart. He is also supporting IIT Madras's AI4Bharat, an [redacted] AI initiative, in 22 Indian languages



Roughly speaking, this task of predicting the next token in a sequence is called language modelling

However, we know that the words in a sentence are not independent but depend on the previous words

a. I enjoyed reading a **book**

b. I enjoyed reading a **thermometer**

The presence of "enjoyed" makes the word "book" more likely than "thermometer"

Hence, the naive assumption does not make sense

$\prod_{i=1}^{T} P(x_i|x_1, \cdots, x_{i-1})$: Current word $\underbrace{x_i}$ depends on previous words $\underbrace{x_1, \cdots, x_{i-1}}$

$\prod_{i=1}^{T} P(x_i|x_1, \cdots, x_{i-1})$: Current word $\underbrace{x_i}$ depends on previous words $\underbrace{x_1, \cdots, x_{i-1}}$



How do we estimate these conditional probabilities?

One solution: use **autoregressive models** where the conditional probabilities are given by parameterized functions with a fixed number of parameters (like transformers).

Causal Language Modelling (CLM)

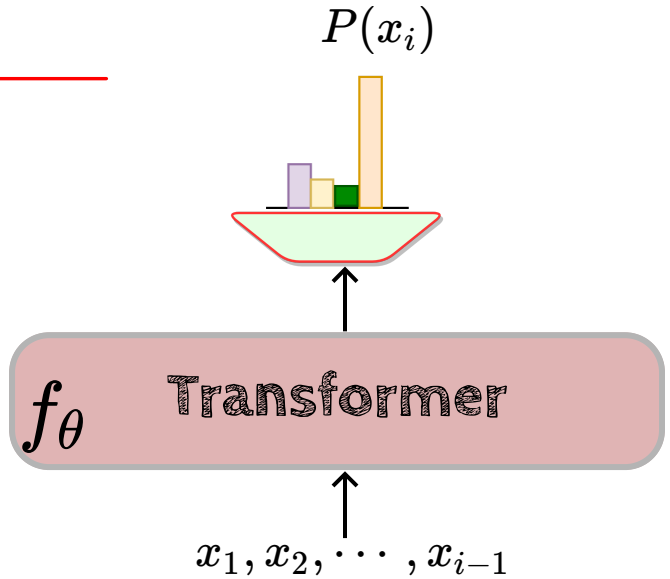
$$P(x_1, x_2, \cdots, x_T) = \prod_{i=1}^{T} P(x_i | x_1, \cdots, x_{i-1})$$

$$= P(x_1)P(x_2|x_1)P(x_3|x_2, x_1) \cdots P(x_T|x_{T-1}, \cdots, x_1)$$

We are looking for f_θ such that

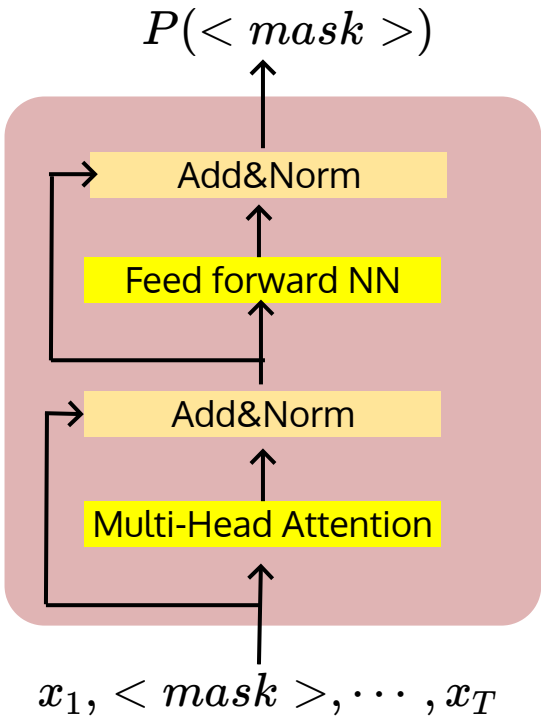
$$P(x_i | x_1, \cdots, x_{i-1}) = f_\theta(x_i | x_1, \cdots, x_{i-1})$$

Can f_θ be a transformer? Yes.



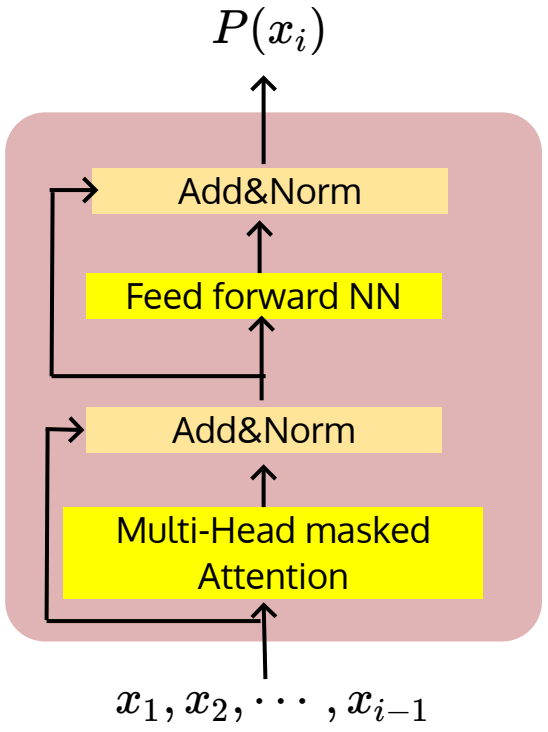
Some Possibilities

BERT

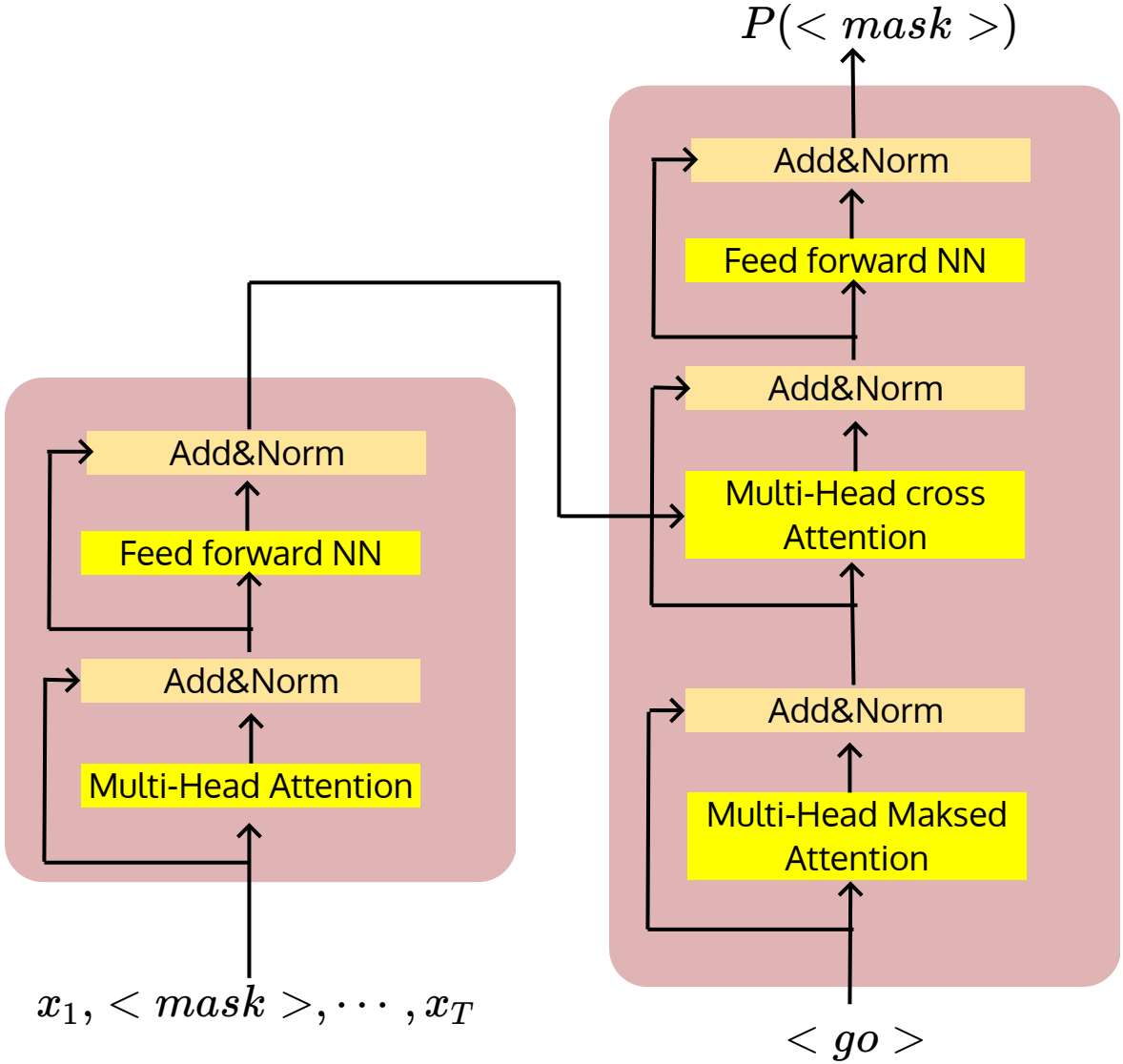


Using only the encoder of the transformer (encoder only models)

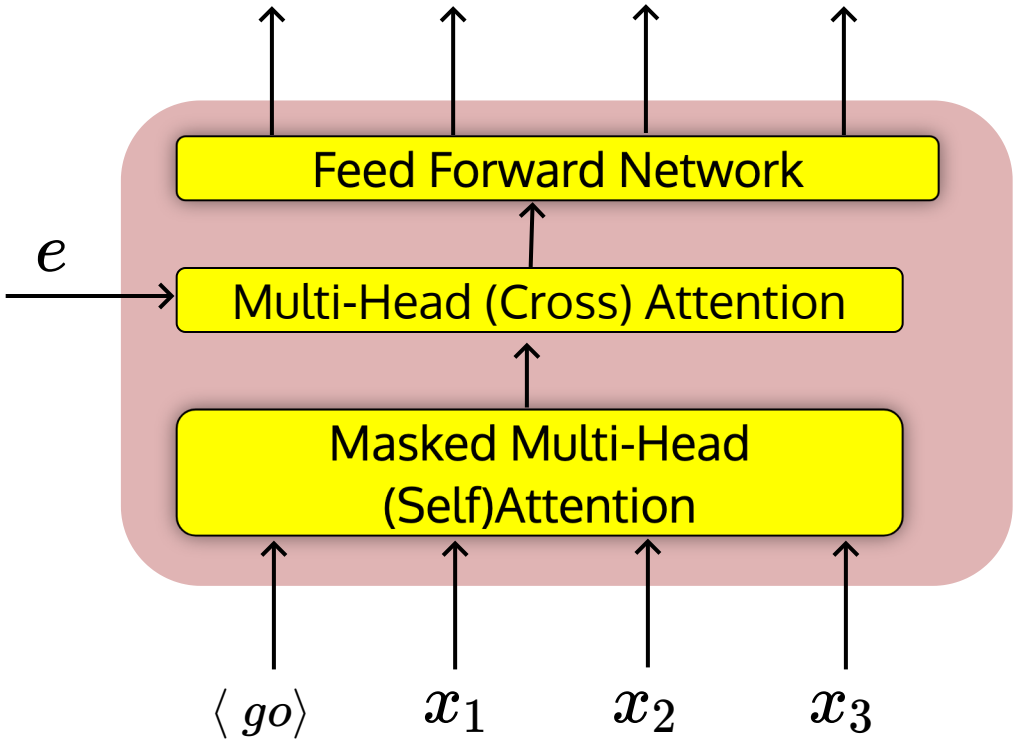
chatGPT



Using only the decoder of the transformer (decoder only models)



Using both the encoder and decoder of the transformer (encoder decoder models)



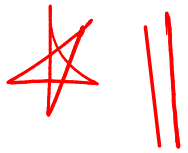
The input is a sequence of words

We want the model to see only the present and past inputs.

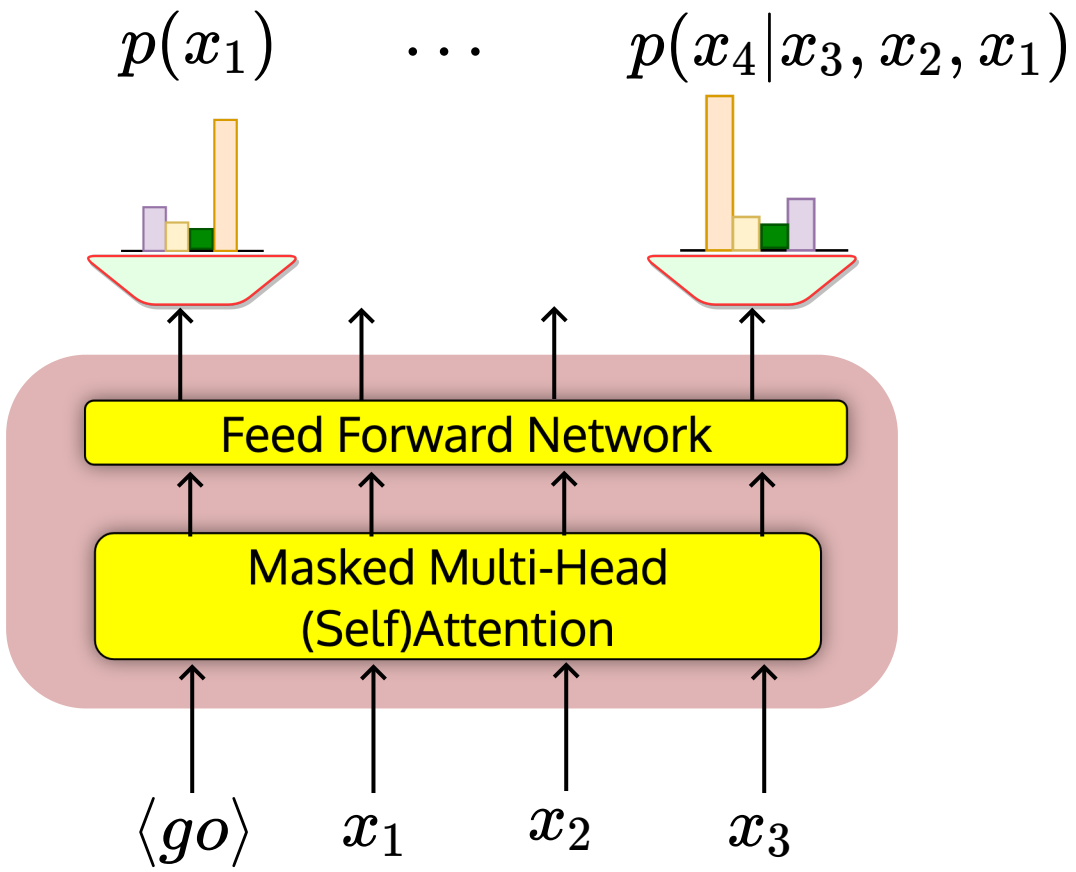
We can achieve this by applying the mask.

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

The masked multi-head attention layer is required.



However, we do not need multi-head cross attention layer (as there is no encoder).



The outputs represent each term in the chain rule

$$= P(x_1)P(x_2|x_1)P(x_3|x_2, x_1)P(x_4|x_3, x_2, x_1)$$

However, this time the probabilities are determined by the parameters of the model,

$$= P(x_1; \theta)P(x_2|x_1; \theta)P(x_3|x_2, x_1; \theta)P(x_4|x_3, x_2, x_1; \theta)$$

Therefore, the objective is to maximize the likelihood $L(\theta)$

Module 2.2 : Generative Pretrained Transformer (GPT)

Mitesh M. Khapra





AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Generative Pretrained Transformer (GPT)

Now we can create a stack (n) of modified decoder layers (called transformer block in the paper)

Let, X denote the input sequence

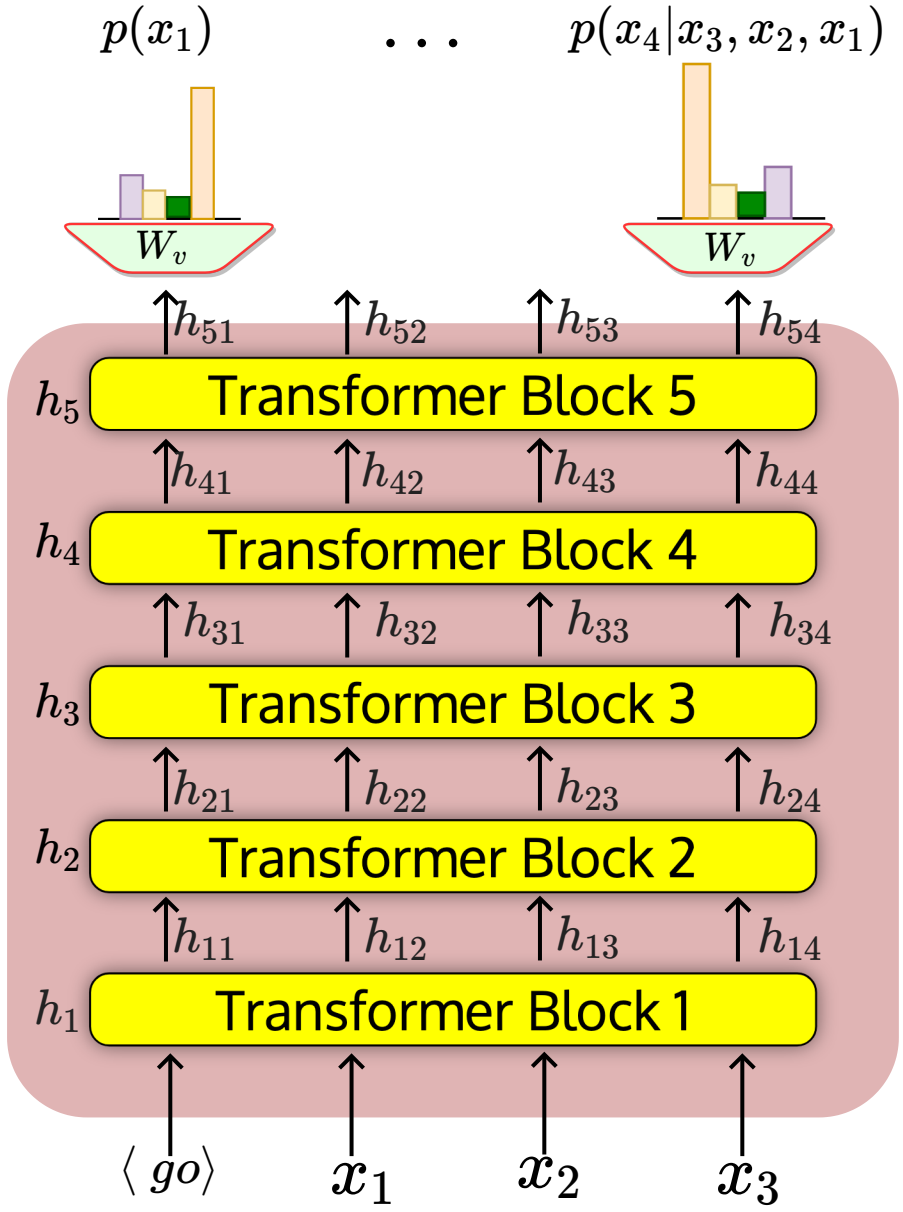
$$h_0 = X \in \mathbb{R}^{T \times d_{model}}$$

$$h_l = transformer_block(h_{l-1}), \forall l \in [1, n]$$

Where $h_n[i]$ is the $i-$the output vector in h_n block.

$$P(x_i) = softmax(h_n[i]W_v)$$

$$\mathscr{L} = \sum_{i=1}^{T} \log(P(x_i|x_1, \cdots, x_{i-1}))$$



Input data

GPT.



BookCorpus

The corpus contains 7000 unique books, 74 Million sentences and approximately 1 Billion words across 16 genres

Also, uses long-range contiguous text

(i.e., no shuffling of sentences or paragraphs)

Tokenizer: Byte Pair Encoding

Vocab size $|\mathcal{V}|$: 40478 ✓

Embedding dim: 768 ✓

Side Note: The other benchmark dataset called 1B words could also be used. However, the sentences are not contiguous (no entailment)

MODEL

Contains 12 decoder layers (transformer blocks)

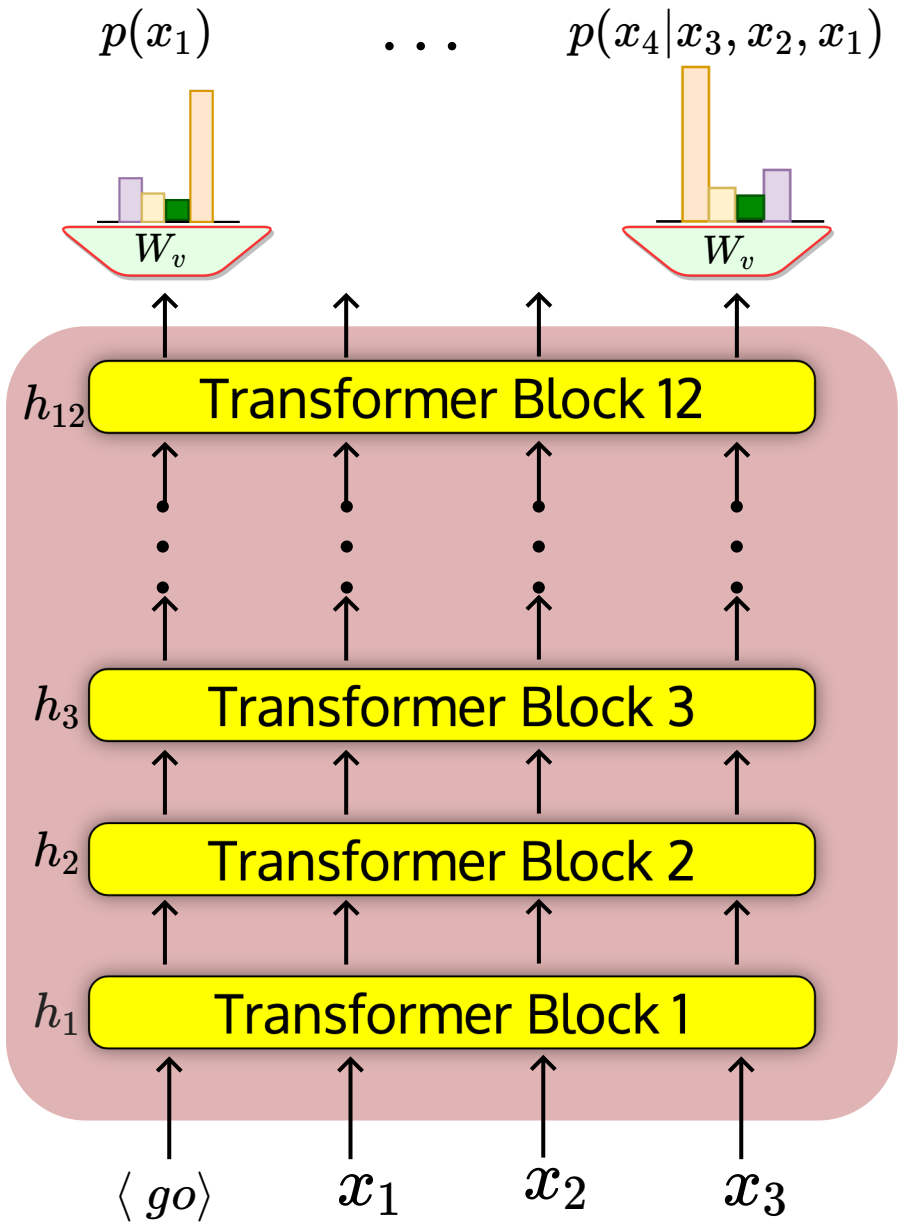
Context size: 512

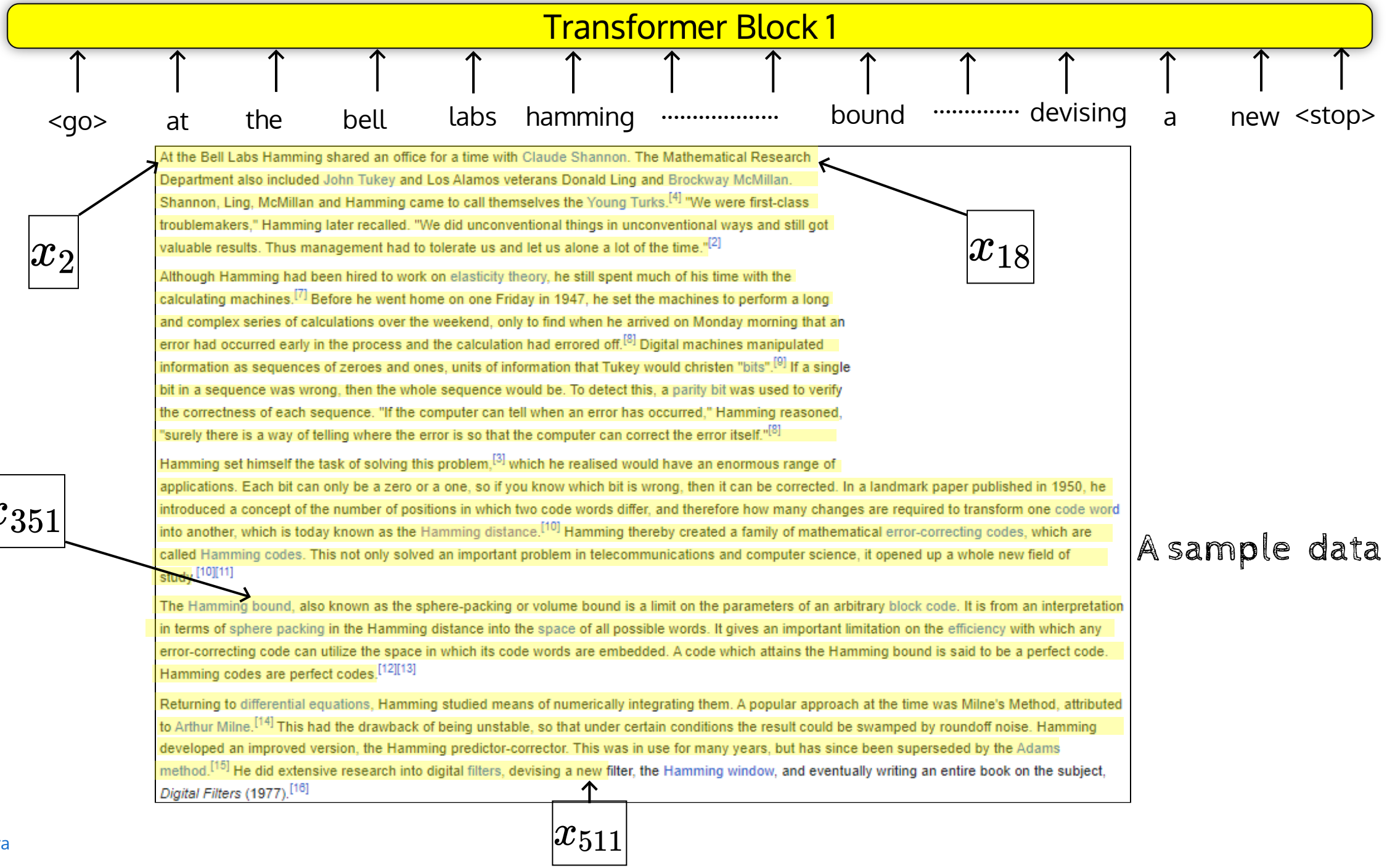
Attention heads: 12

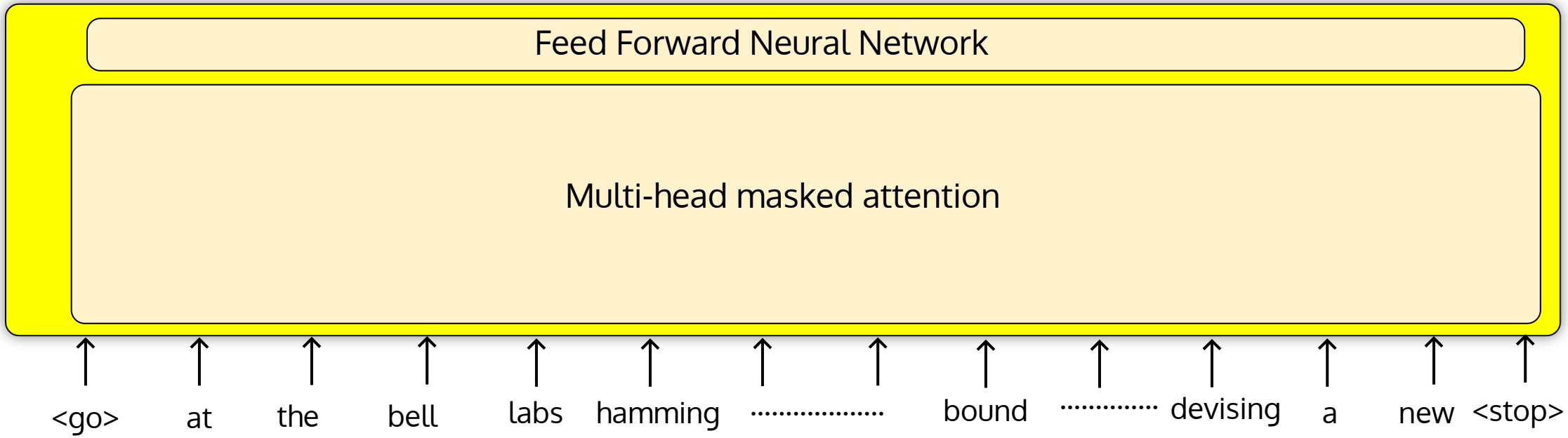
FFN hidden layer size: $768 \times 4 = 3072$

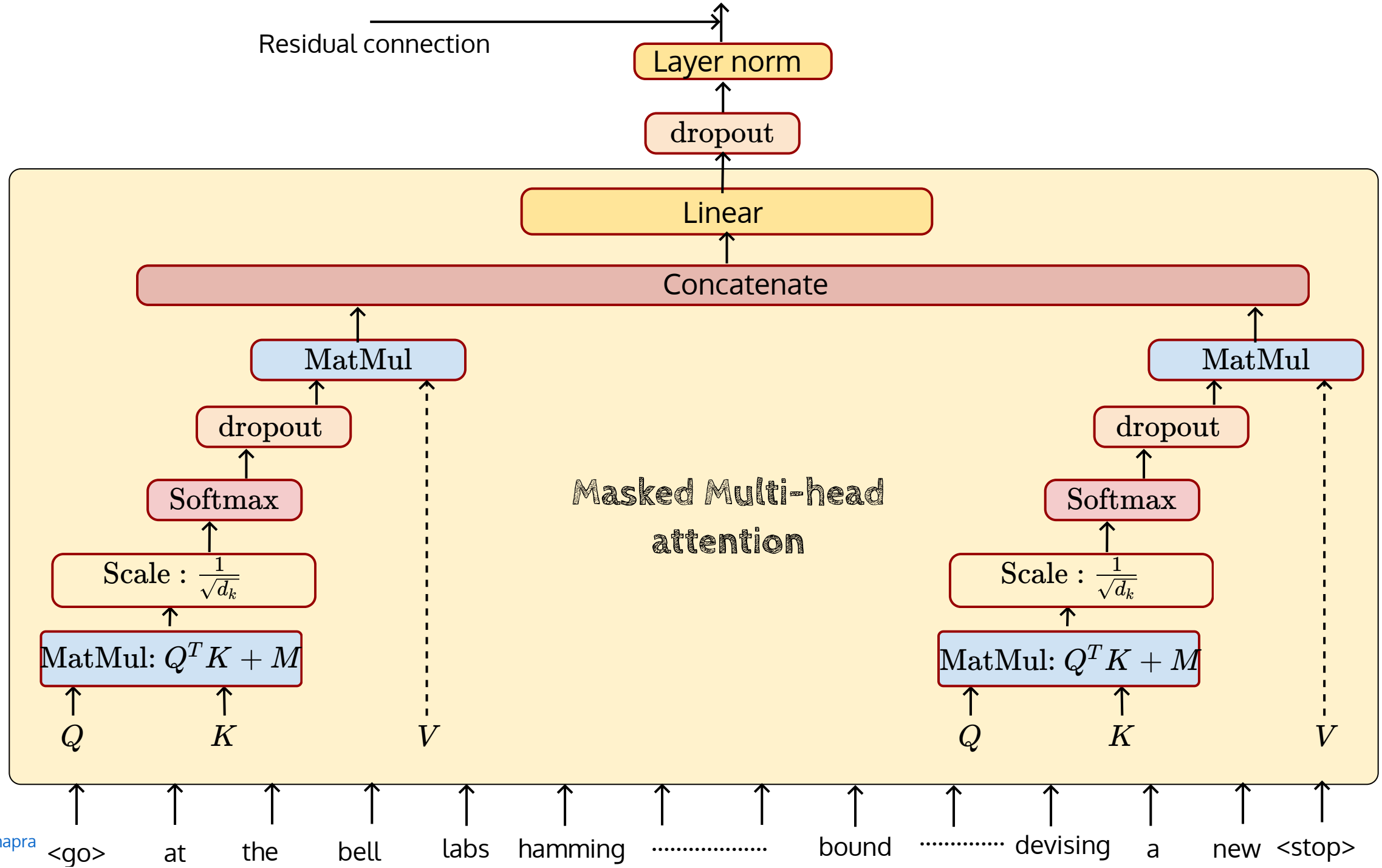
Activation: Gaussian Error Linear Unit (GELU)

Dropout, layer normalization, and residual connections were implemented to enhance convergence during training.




Transformer Block 1
<go>
at
the
bell
labs
hamming
bound
devising
a
new
<stop>
x_2
x_{18}
x_{351}
x_{511}
At the Bell Labs Hamming shared an office for a time with Claude Shannon. The Mathematical Research Department also included John Tukey and Los Alamos veterans Donald Ling and Brockway McMillan. Shannon, Ling, McMillan and Hamming came to call themselves the Young Turks.[4] "We were first-class troublemakers," Hamming later recalled. "We did unconventional things in unconventional ways and still got valuable results. Thus management had to tolerate us and let us alone a lot of the time."[2]
Although Hamming had been hired to work on elasticity theory, he still spent much of his time with the calculating machines.[7] Before he went home on one Friday in 1947, he set the machines to perform a long and complex series of calculations over the weekend, only to find when he arrived on Monday morning that an error had occurred early in the process and the calculation had errored off.[8] Digital machines manipulated information as sequences of zeroes and ones, units of information that Tukey would christen "bits".[9] If a single bit in a sequence was wrong, then the whole sequence would be. To detect this, a parity bit was used to verify the correctness of each sequence. "If the computer can tell when an error has occurred," Hamming reasoned, "surely there is a way of telling where the error is so that the computer can correct the error itself."[8]
Hamming set himself the task of solving this problem,[3] which he realised would have an enormous range of applications. Each bit can only be a zero or a one, so if you know which bit is wrong, then it can be corrected. In a landmark paper published in 1950, he introduced a concept of the number of positions in which two code words differ, and therefore how many changes are required to transform one code word into another, which is today known as the Hamming distance.[10] Hamming thereby created a family of mathematical error-correcting codes, which are called Hamming codes. This not only solved an important problem in telecommunications and computer science, it opened up a whole new field of study.[10][11]
The Hamming bound, also known as the sphere-packing or volume bound is a limit on the parameters of an arbitrary block code. It is from an interpretation in terms of sphere packing in the Hamming distance into the space of all possible words. It gives an important limitation on the efficiency with which any error-correcting code can utilize the space in which its code words are embedded. A code which attains the Hamming bound is said to be a perfect code. Hamming codes are perfect codes.[12][13]
Returning to differential equations, Hamming studied means of numerically integrating them. A popular approach at the time was Milne's Method, attributed to Arthur Milne.[14] This had the drawback of being unstable, so that under certain conditions the result could be swamped by roundoff noise. Hamming developed an improved version, the Hamming predictor-corrector. This was in use for many years, but has since been superseded by the Adams method.[15] He did extensive research into digital filters, devising a new filter, the Hamming window, and eventually writing an entire book on the subject, Digital Filters (1977).[16]
A sample data


Feed Forward Neural Network
Multi-head masked attention
<go>
at
the
bell
labs
hamming
.....
bound
.....
devising
a
new
<stop>



Mitesh Khapra

Residual connection

Layer norm

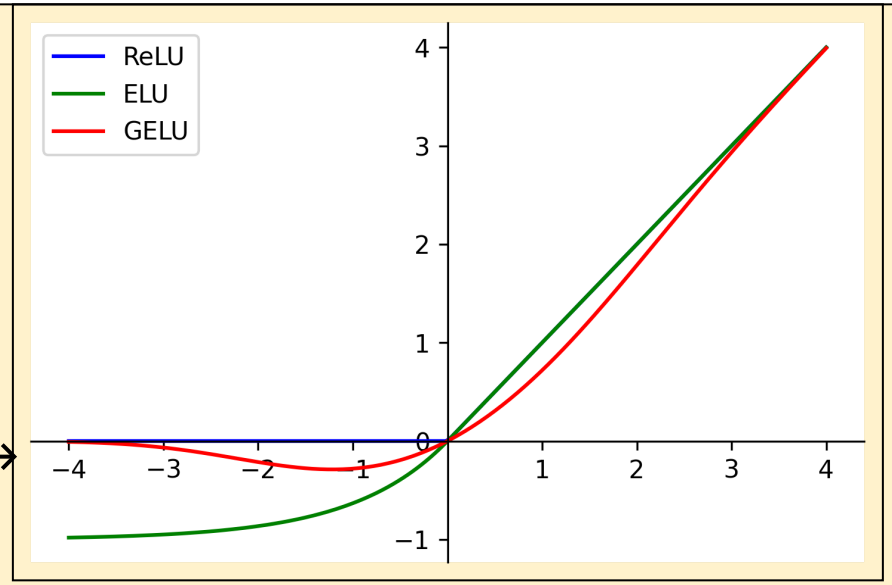
// Pre - Norm

dropout

$o_1 = \mathbb{R}^{768}$

$l = 3042$

$z_1 = \mathbb{R}^{768}$



Feed Forward Neural Network

Masked Multi-head attention

<go> at the bell labs hamming ..... bound ..... devising a new <stop>

Mitesh Khapra

Number of parameters

token Embeddings: $|\mathcal{V}| \times$ embedding_dim

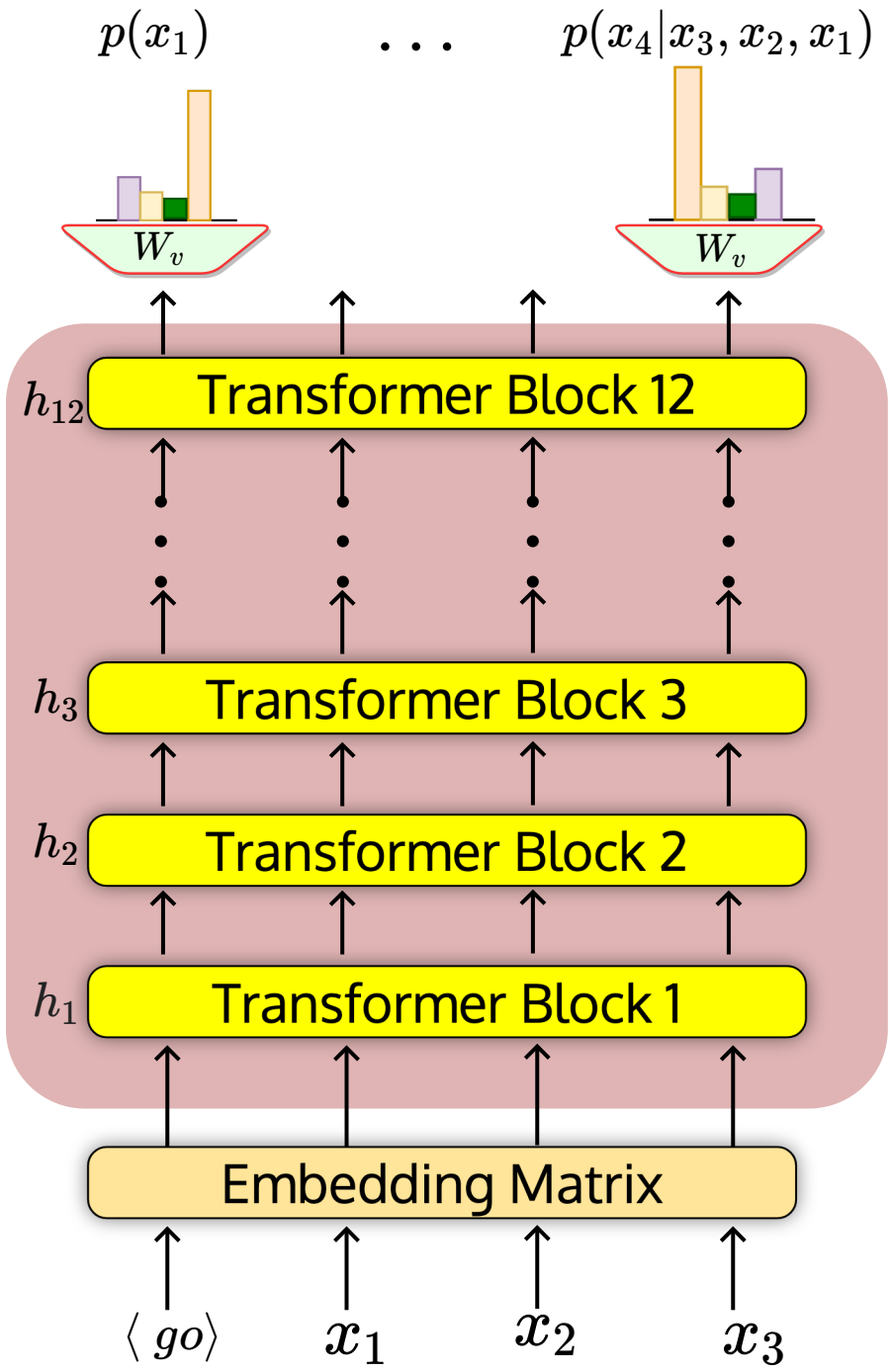
$$40478 \times 768 = 31 \times 10^6 = 31M$$

Position Embeddings : context length $\times$ embedding_dim

$$512 \times 768 = 0.3 \times 10^6 = 0.3M$$

Total: $31.3M$

The positional embeddings are also learned, unlike the original transformer which uses fixed sinusoidal embeddings to encode the positions.



Number of parameters

768 / 12 → no. of heads.

Attention parameters per block

$$W_Q = W_K = W_v = (768 \times 64)$$

Per attention head

$$3 \times (768 \times 64) \approx 147 \times 10^3$$ ✓

For 12 heads

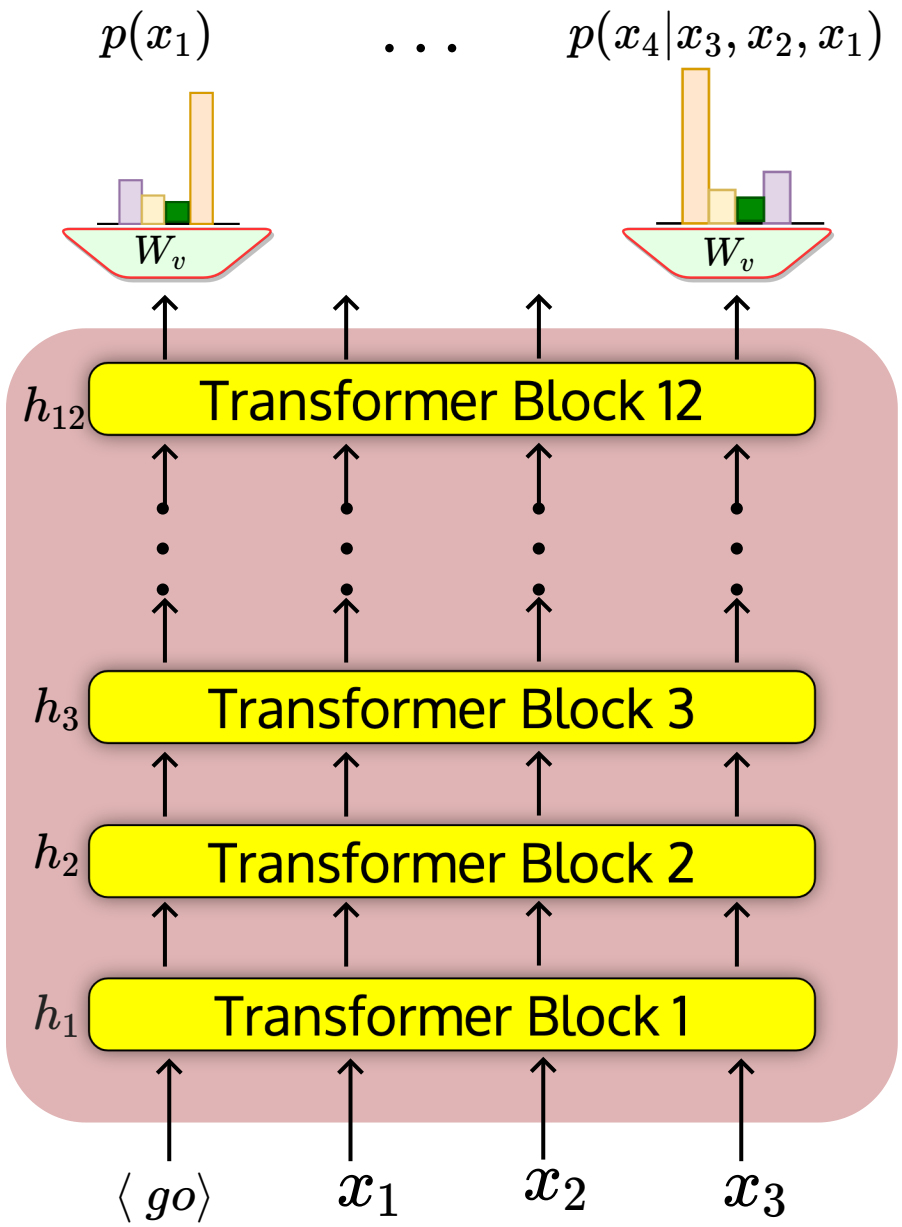
$$12 \times 147 \times 10^3 \approx 1.7M$$

4 × 768 × 768 = 23,59,296

For a Linear layer: kl_0 • = $768 \times 768 \approx 0.6M$

For all 12 blocks // layers • 12 = 2,83,11,552

$$12 \times 2.3 = 27.6M$$



Number of parameters

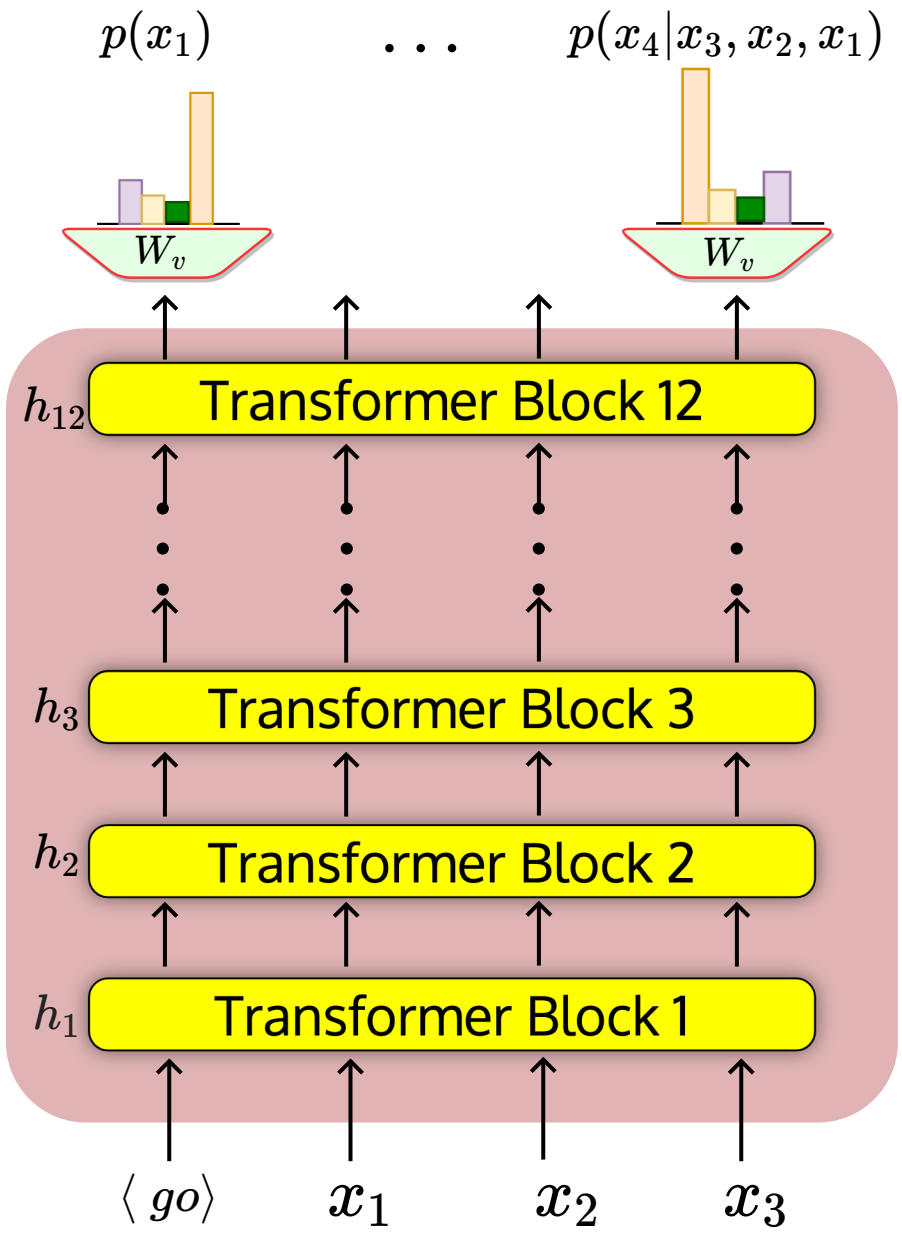
FFN parameters per block

bias1 bias 2.

$$2 \times (768 \times 3072) + 3072 + 768$$
$$= 4.7 \times 10^6 = 4.7M$$

For all 12 blocks

$$12 \times 4.7 = 56.4M$$

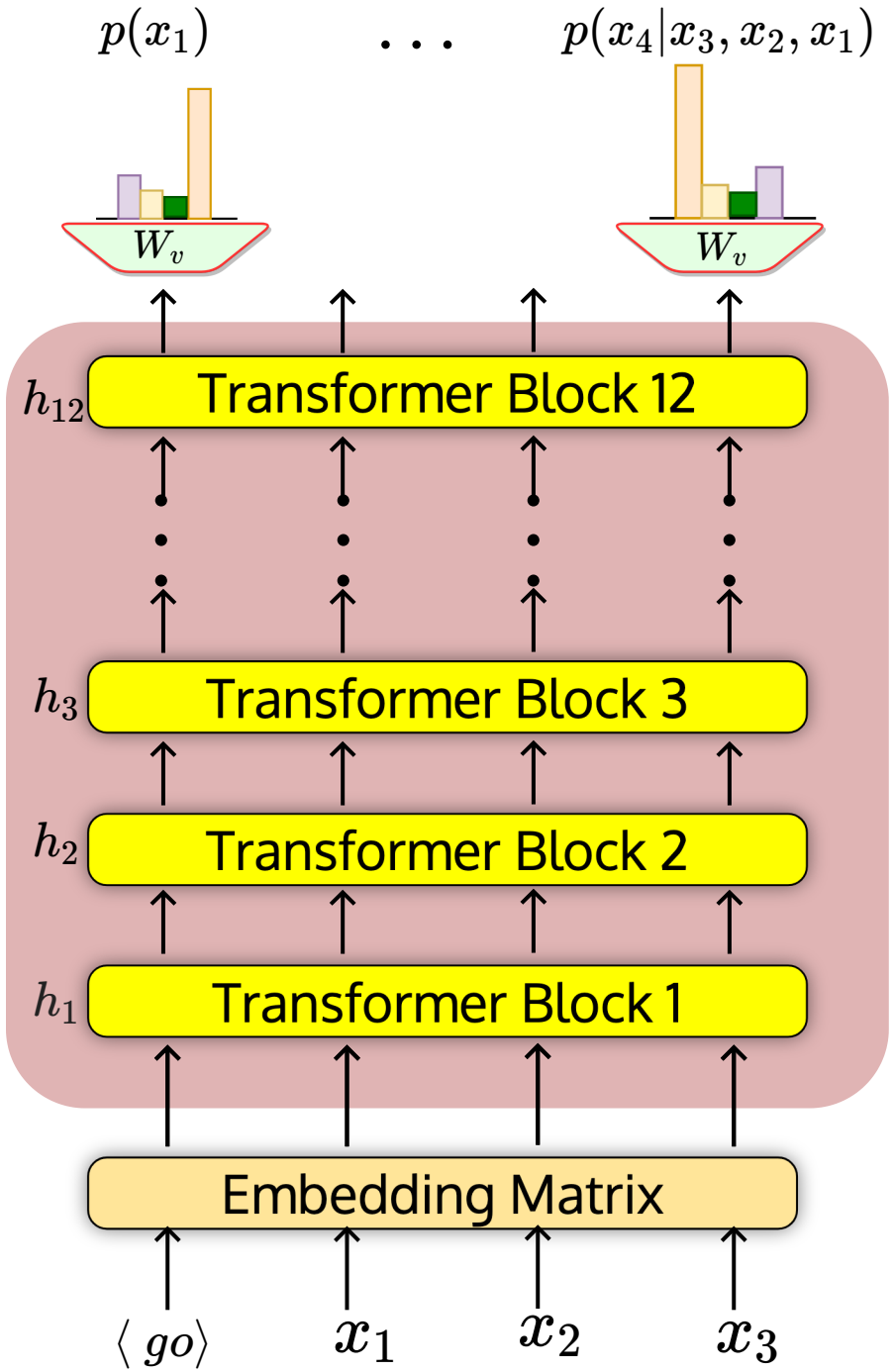


Number of parameters

Layer	Parameters (in Millions)
Embedding Layer	31.3
Attention layers	27.6
FFN Layers	56.4
Total	116.461056*

Thus, GPT-1 has around 117 million parameters.

*Without rounding the number of parameters in each layer



Mesh Khana

Module 2.3 : Pre-training and Fine Tuning

Mitesh M. Khapra





AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Pre-Training

Batch Size: 64

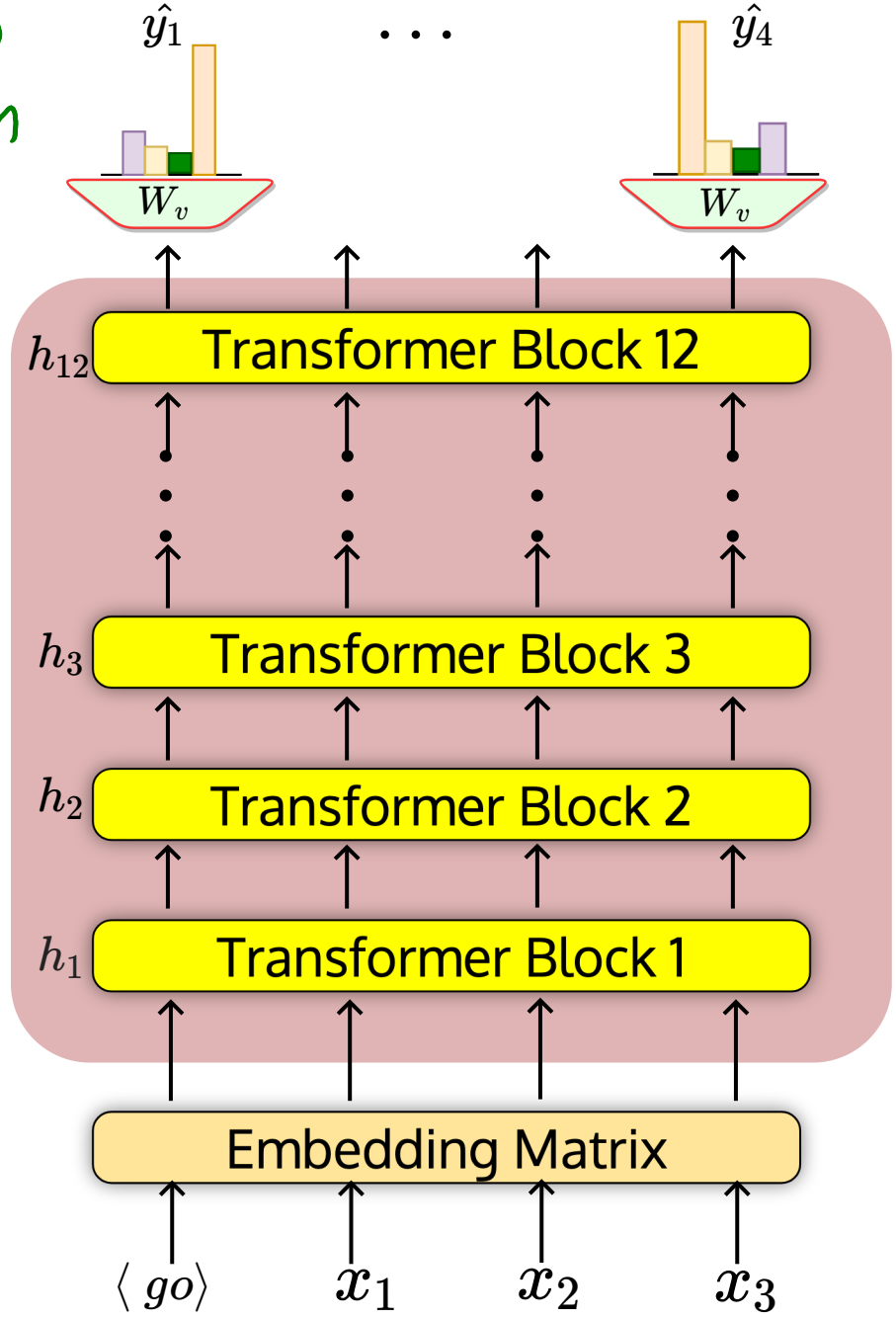
Input size: $(B, T, C) = (64, 512, 768)$, where, C is an embedding dimension

Minimize

$$\mathscr{L} = -\sum_{x \in V : X \in \mathcal{X}} \sum_{i=1}^{T} y_i \log(\hat{y}_i))$$

Optimizer: Adam with cosine learning rate scheduler

Strategy : Teacher forcing (instead of auto-regressive training) for quicker and stable convergence



Fine-tuning

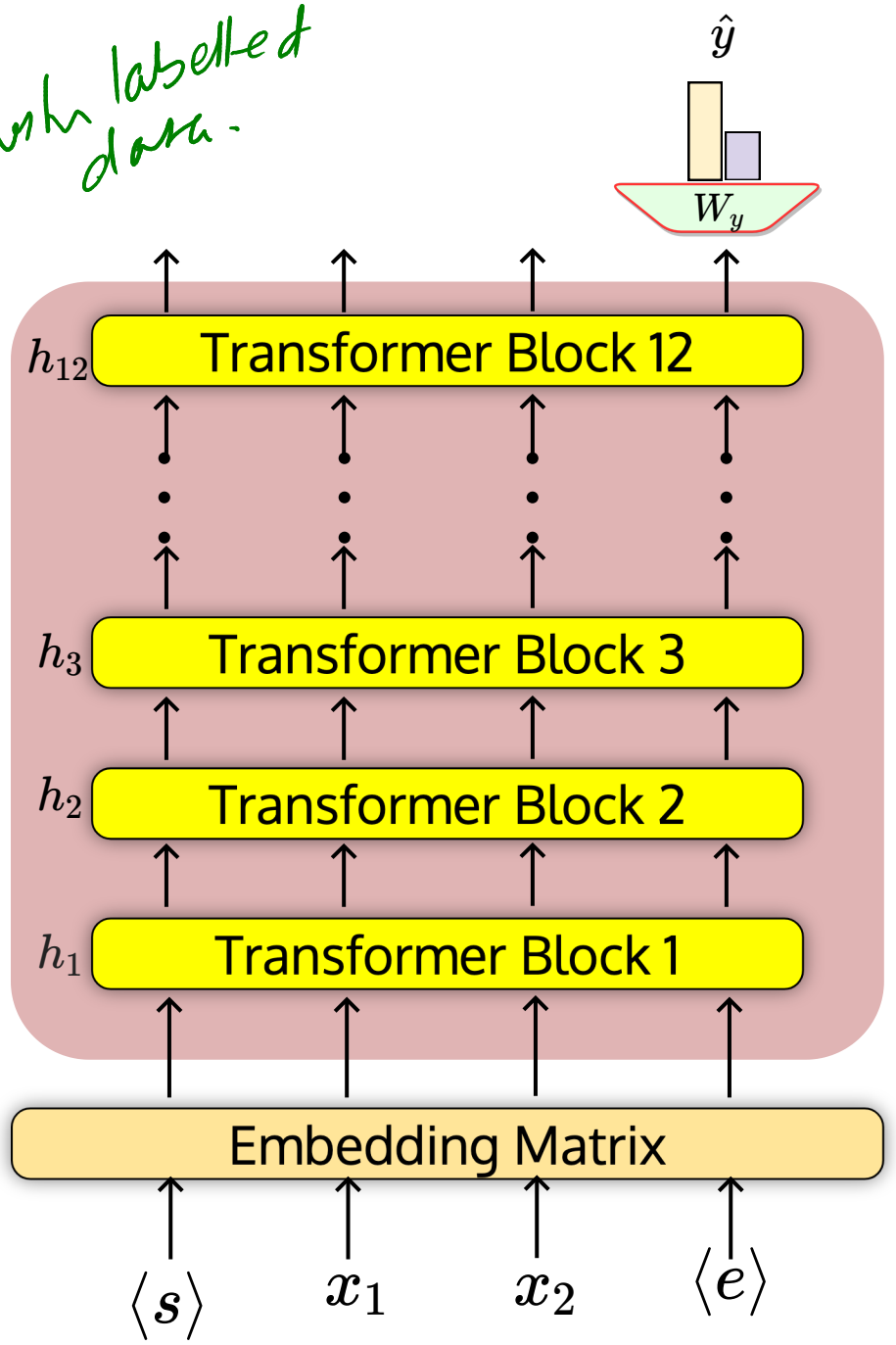
Fine-tuning involves adapting model for various downstream tasks (with a minimal change in the architecture)

Each sample in a labelled data set $\mathcal{C}$ consists of a sequence of tokens $x_1, x_2, \cdots, x_m$ with the label y

Initialize the parameters with the parameters learned by solving the pre-trianing objective

At the input side, add additional tokens based on the type of downstream task. For example, start $\langle s \rangle$ and end $\langle e \rangle$ tokens for classification tasks

At the output side, replace the pre-training LM head with the classification head (a linear layer W_y)



Fine-tuning

Now our objective is to predict the label of the input sequence

$$\hat{y} = P(y|x_1, \cdots, x_m) = softmax(W_y h_l^m)$$

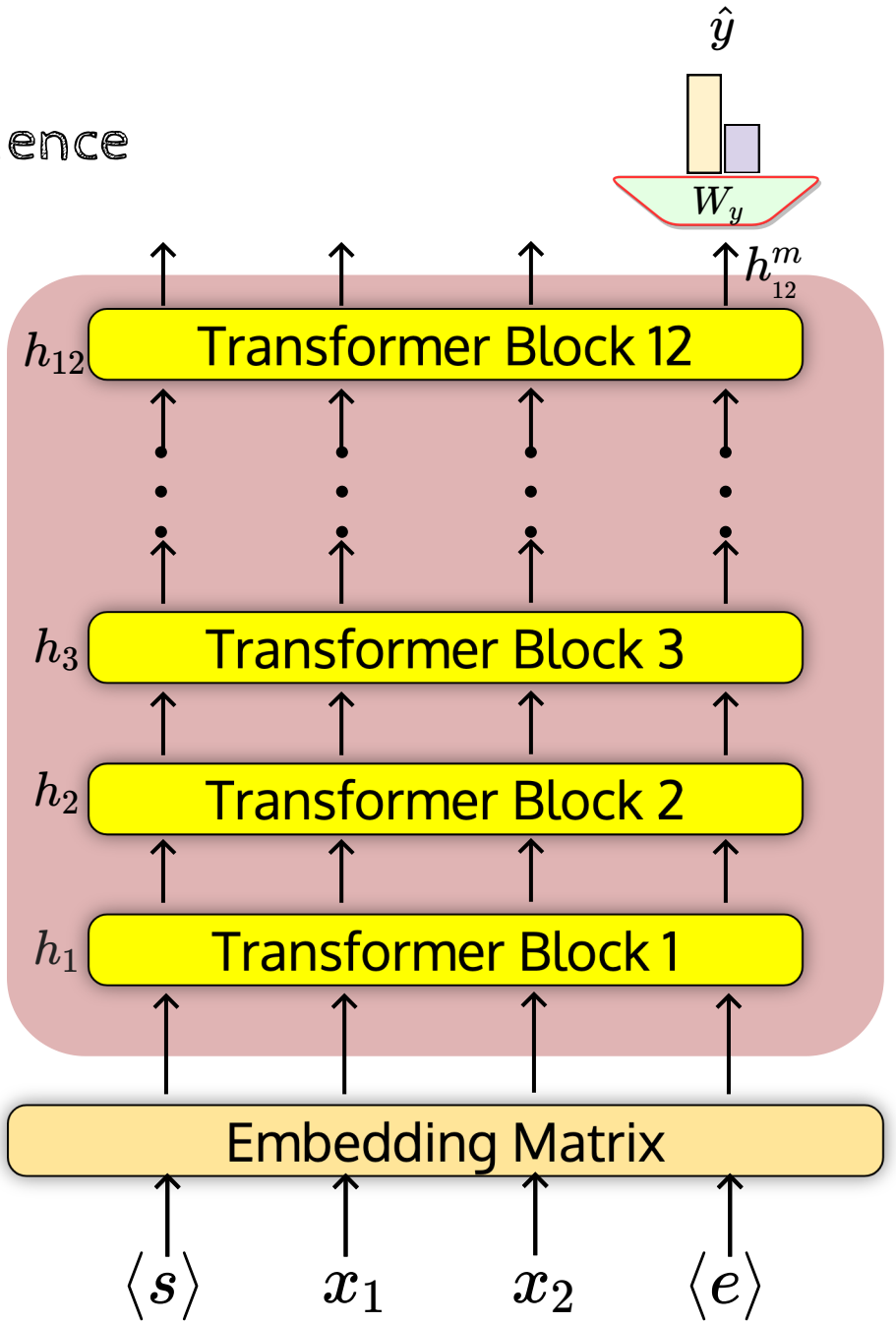
Note that we take the output representation at the last time step of the last layer h_l^m.

It makes sense as the entire sentence is encoded only at the last time step due to causal masking.

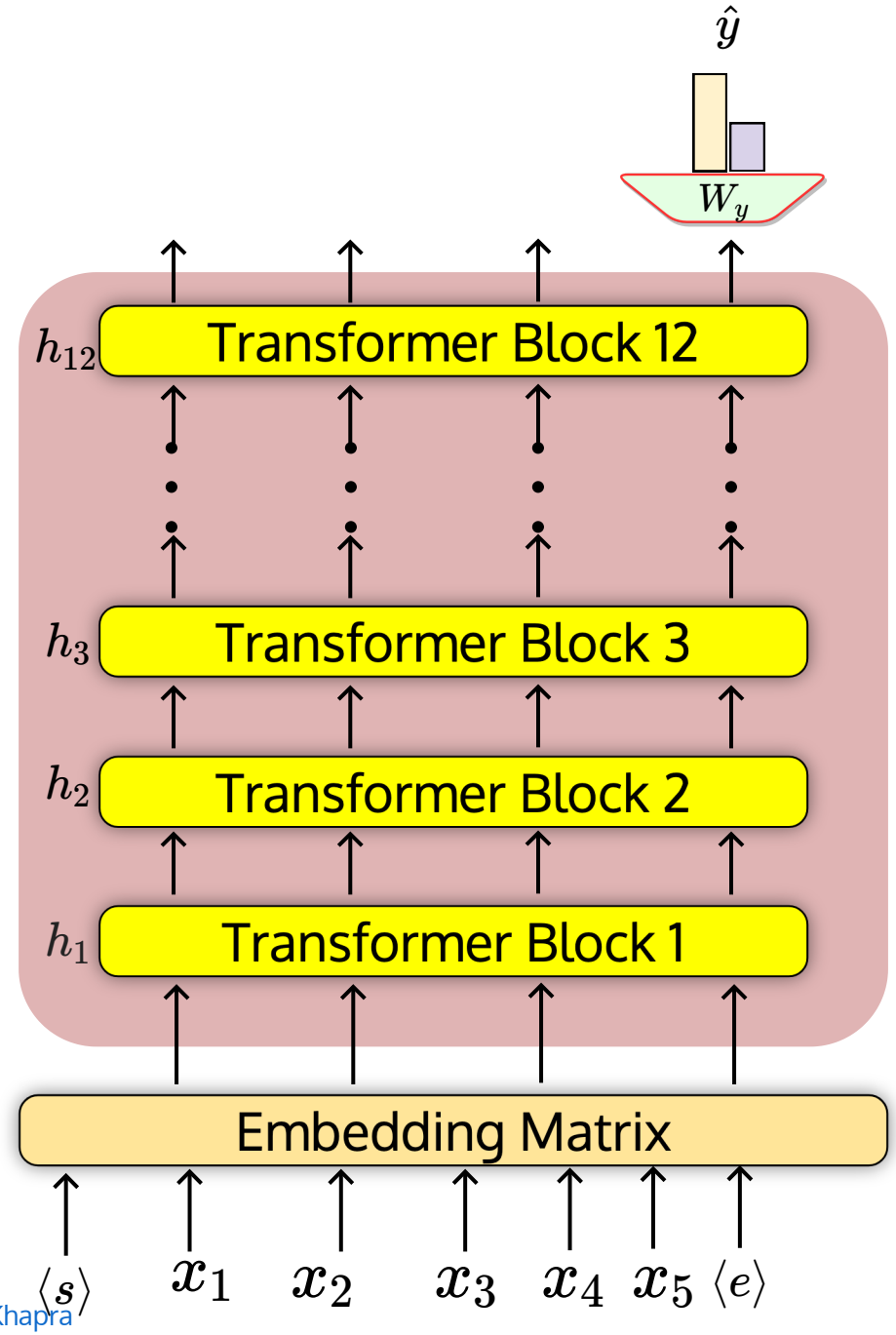
Then we can minimize the following objective

$$\mathscr{L} = -\sum_{(x,y)} \log(\hat{y}_i)$$

Note that W_y is randomly intialized. Padding or truncation is applied if the length of input sequence is less or greater than the context length



Sentiment Analysis

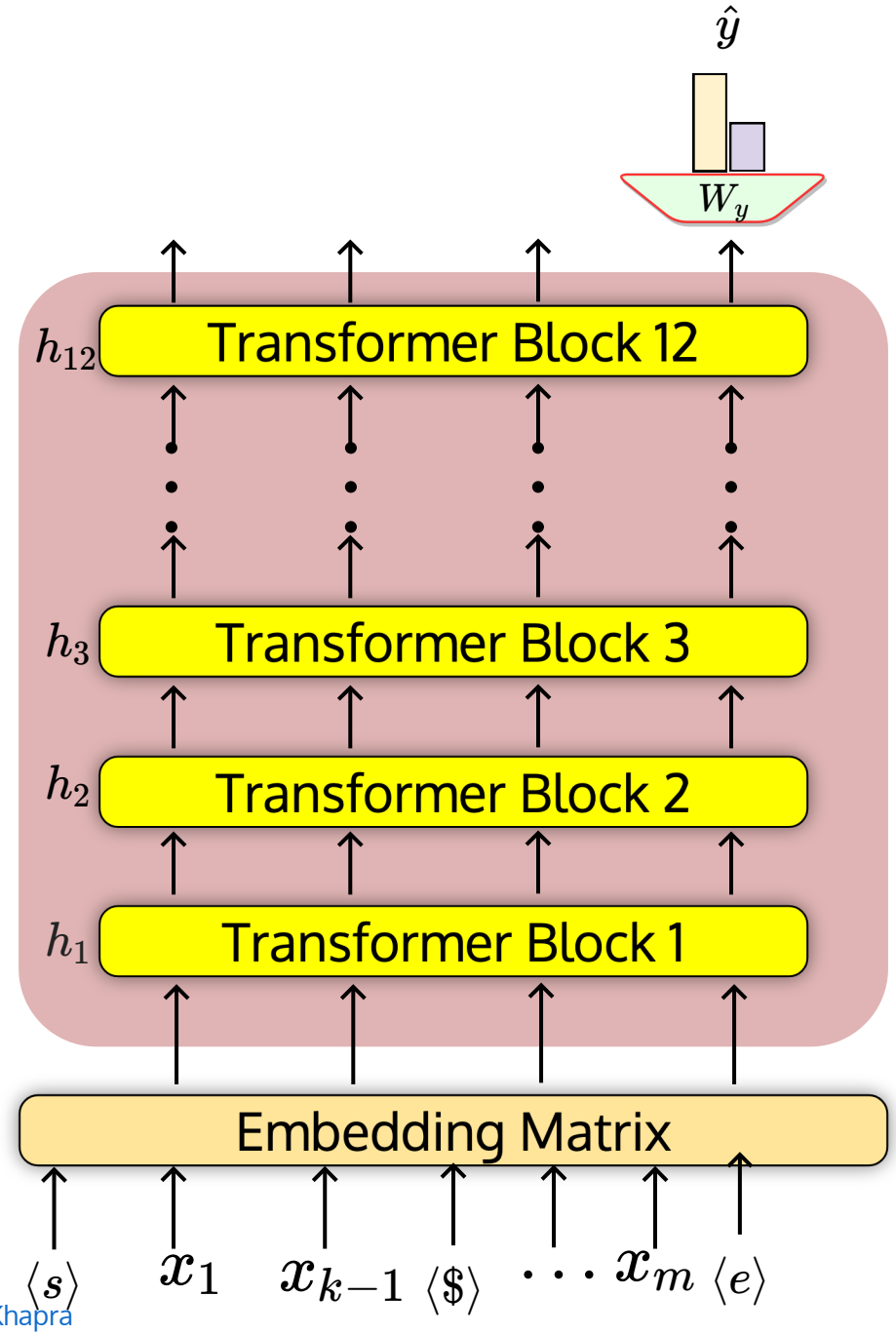


Text: Wow, India has now reached the moon

Sentiment: Positive

$$\hat{y} \in \{+1, -1\}$$

- We can train all parameters.
- Or freeze pre-trained parameters (called freezing)
- Or train parameters upto 'k' layers only (Partial freezing).



Textual Entailement/Contradiction

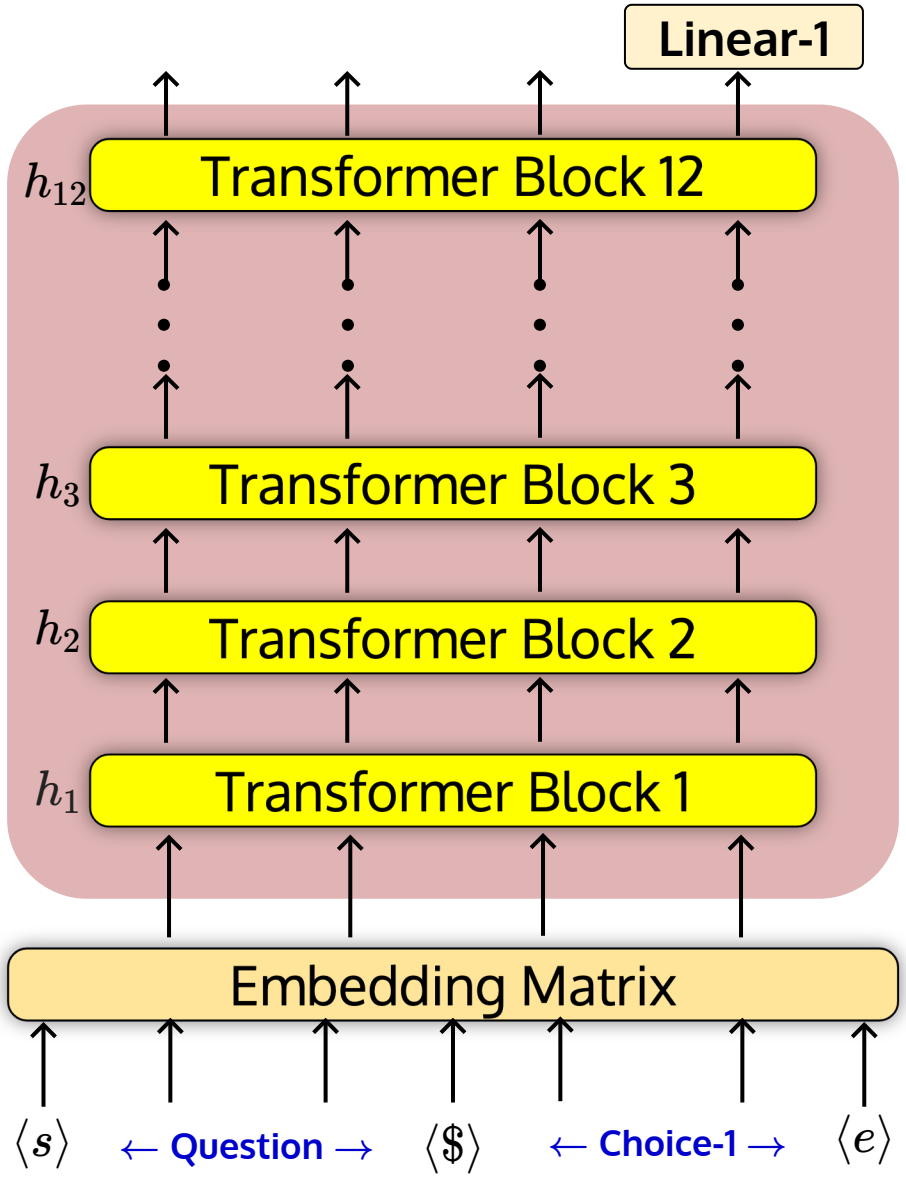
Text: A soccer game with multiple males playing

Hypothesis: Some men are playing a sport

Entailment: True

In this case, we need to use a delimiter token $\langle \$ \rangle$ to differentiate the text from the hypothesis.

Multiple Choice

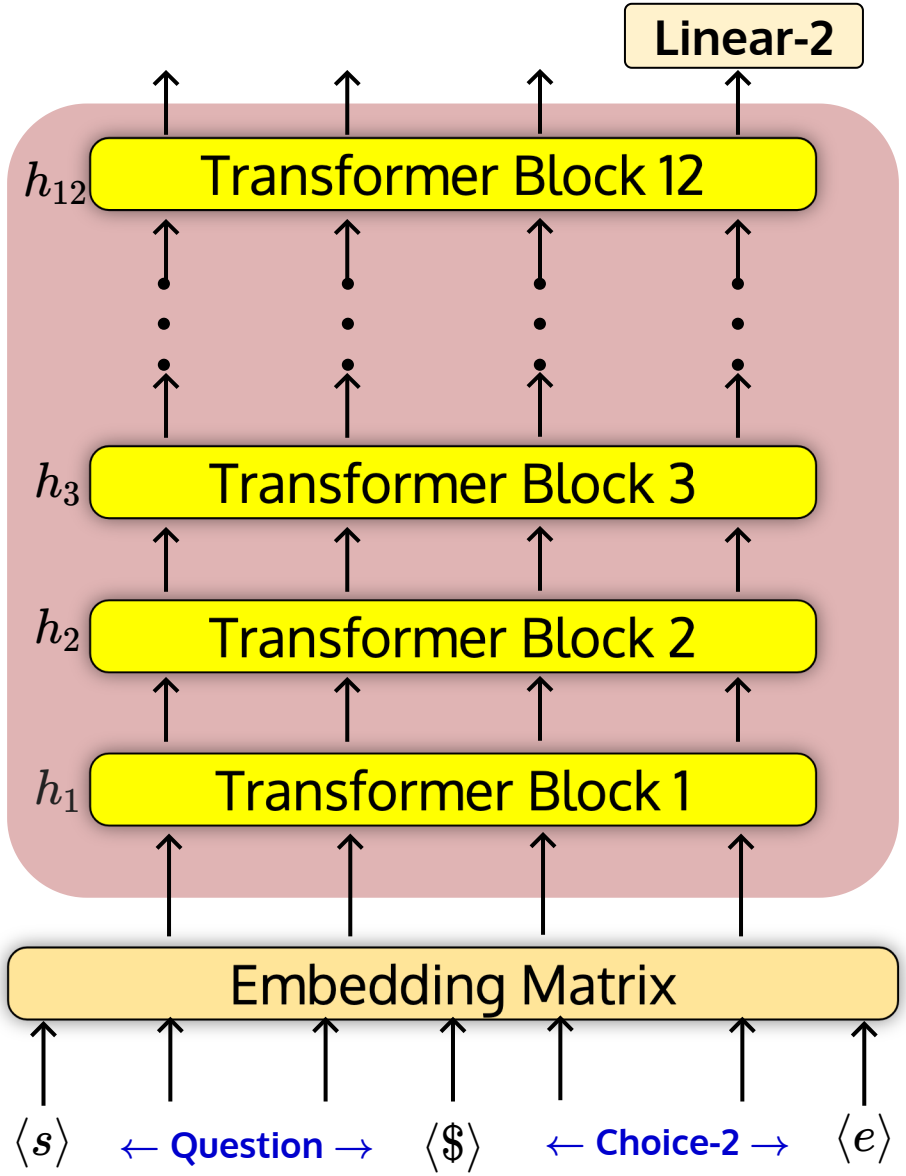


Question: Which of the following animals is an amphibian?

Choice: Frog

Choice: Fish

Feed in the question along with the choice-1



Multiple Choice

Question: Which of the following animals is an amphibian?

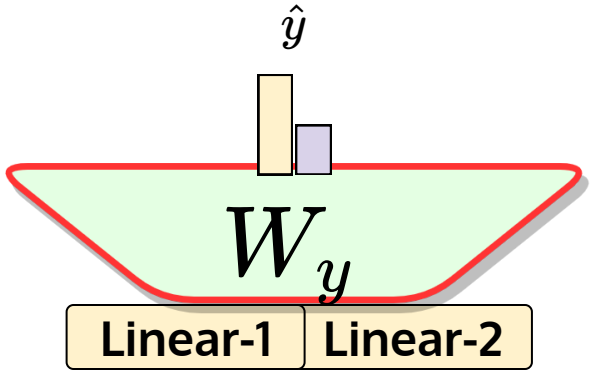
Choice: Frog

Choice: Fish

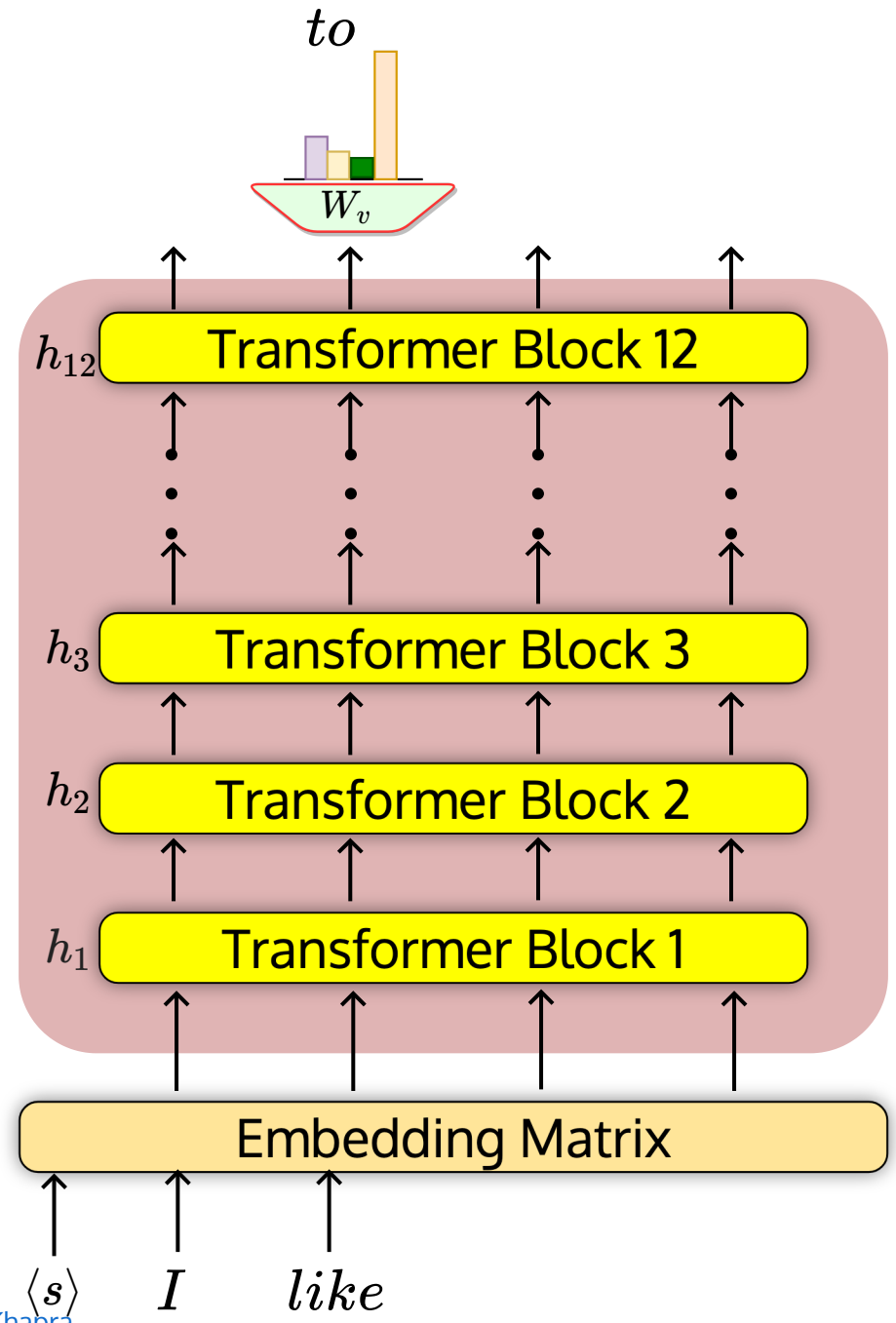
Feed in the question along with the choice-2

Repeat this for all choices

Normalize via softmax



Text Generation



Input:

Prompt: I like

$$M = \begin{bmatrix} 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \end{bmatrix}$$

Feed in the prompt along with the mask and run the model in autregressive mode

Stoping Criteria:

Sequence length: 5

or outputing a token: ‹e›

Output: I like to think that

Does it produce the same output sequence for the given prompt?

or Will it be creative ?

Wishlist for text generation

Discourage degenerative (that is, repeated or incoherent) texts

I like to think that I like to think...

I like to think that reference know how to think best selling book

Encourage it to be Creative in generating a sequence for the same prompt

I like to read a book

I like to buy a hot beverage

I like a person who cares about others

Accelarate the generation of tokens

Wishlist for text generation

Discourage degenerative (that is, repeated or incoherent) texts

I like to think that I like to think...

I like to think that reference know how to think best selling book

Encourage it to be Creative in generating a sequence for the same prompt

I like to read a book

I like to buy a hot beverage

I like a person who cares about others

Accelarate the generation of tokens

GAS

1) Consider the embedding vector for a word, $x = [0.1, 0, 0.23, 0.4, -0.75, 1]$. Suppose the word is at position 4 in the given sentence. Add the corresponding position embedding p to the word embedding and enter the sum of the elements in $h = x + p$.Use the fixed-sinusoidal position embedding vector calculated using the formula given below

$$PE(pos, 2i) = sin\left(\frac{pos}{10000^{2i/dmodel}}\right)$$

$$PE(pos, 2i+1) = cos\left(\frac{pos}{10000^{2i/dmodel}}\right)$$

embedding size → 6 ↓ 4th word.

Example I am not going home today

$d_{model} = 6$

$PE(4, 0) = \sin\left(\frac{4}{10000^{0}}\right) = \sin 4 = -0.75$

$PE(4, 1) = \cos\left(\frac{4}{10000^{0}}\right) = -0.65$

$PE(4, 2) = \sin\left(\frac{4}{10000^{2/6}}\right) = 0.18$

$PE(4, 3) = \cos\left(\frac{4}{10000^{2/6}}\right) = 0.98$

$PE(4, 4) = \sin\left(\frac{4}{10000^{4/6}}\right) = 0.00$

$PE(4, 5) = \cos\left(\frac{4}{10000^{4/6}}\right) = 0.99$

0.1
0
0.23
0.4
-0.75
1

→ embedding

$\therefore x = h + p = (-0.65, -0.65, 0.41, 1.38, -0.75, 1.99)$

Sum = 3.78 − 2.05 = 1.73 ✓

⚠ Remember $i \in \left[0, \frac{dmodel - 2}{2}\right]$

$\therefore i = 0, 1, 2.$

use $\sin\left(\frac{Pos}{10000^{2i/dmodel}}\right)$ & $\cos\left(\frac{Pos}{10000^{2i/dmodel}}\right)$

0 — So	$\sin \frac{4}{10000^{0}}$
1 —	$\cos \frac{4}{10000^{0}}$
2 —	$\sin \frac{4}{10000^{2/6}}$
3 —	$\cos \frac{4}{10000^{2/6}}$
4 —	$\sin \frac{4}{10000^{4/6}}$
5 —	$\cos \frac{4}{10000^{4/6}}$

[0.1, 0, 0.23, 0.4, −0.75, 1]

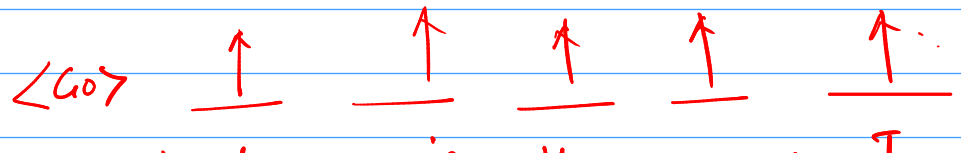
```
[-7.57e-01  9.83e-01  8.62e-03  1.00e+00  1.86e-05  1.00e+00]
Word embedding: [ 0.1   0.    0.23  0.4  -0.75  1.  ]
Positional embedding: [-7.57e-01  9.83e-01  8.62e-03  1.00e+00  1.86e-05  1.00e+00]
Combined embedding (h): [-0.66  0.98  0.24  1.4  -0.75  2.  ]
Sum of elements in h: 3.2146475990476446
```

Sum =

2) Consider a language where sentences are written from Right-to-Left (RTL) direction instead of Left-to-Right writing (reading) method. An example sentence is ``transformer movie the enjoyed I". Suppose we decided to use the GPT for Causal Language Modelling (CLM) of the text, then

transformer movie the enjoyed I

- It is necessary to flip the attention mask horizontally
- It is necessary to flip the attention mask vertically
- ✓ Neither the horizontal nor the vertical flipping of attention mask is required



∴ No requirement to change the mask.

3) Choose the correct statements

- ☐ All the parameters of the GPT model are randomly initialized during pre-training Yes.
- ☐ The model minimizes the CLM objective during fine-tuning to improve the performance No → It happens during pre-trg
- ☐ All parameters of the GPT model are randomly initialized during fine-tuning No → pre-trained parameters
- ☐ In general, fine-tuning requires a dataset with labels Yes. ✓

4) Assume that we have a large corpus of text. The vocabulary constructed from the text contains 10000 words. Of these, 100 words occurred only once in the entire corpus of text. The parameters of the embedding layer and the output layer of the model are shared. Suppose we create a batch of 256 samples (each sample is a sentence from the corpus). None of these samples contains any of the 100 rare words. Suppose we pre-train the model for one iteration using the batch of samples, then

- it is certain that the embeddings of none of these 100 rare words will get updated.
- ✓ there is a chance that the embeddings of all or some of these 100 rare words will get updated
- the embeddings of all these 100 rare words will defintely get updated

VocabSize → 10000, 256 samples without "100" words.

∴ for this words, embedding will not be avl.

∴

However at the output we have same embedding layer.

When loss flows back, this layer must be updated ∴ Parameters of these 100 words also may get updated.

∴ Second option is correct.

Consider. Output is probability distribution over all 10000 words. & Say probability of these 100 words is not '0'. then the weights in the shared embedding layer will get updated (in case of not '0').

Rajesh has created a GPT-like transformer model. However he doesn't have access to large computation power, so he has configured the original architecture in the following manner:

1. He has taken a vocabulary of size 1000 words/tokens.
2. Sequence length is 64.
3. Embedding dimension and d_{model} is 128.
4. There are only 4 transformer blocks (layers).
5. Each transformer block (layer) has only 4 attention heads.
6. FFN hidden layer size is 256.

embedding_dim = 64 × 128.

∴ posthan_embedy dim → 64 × 128 (Same as embedd. dim)

5) What will be the shape of positional embedding?

- [x] 64×128
- [ ] 64×4
- [ ] 12×128
- [ ] 512×768

6) What will be the shape of W_Q^i?

W_Q = 128 × $\frac{128}{4}$ = 128 × 32.

- [ ] 32×32
- [ ] 64×32
- [ ] 128×16
- [x] 128×32
- [ ] 8×32
- [ ] None of these.

$Q = H \cdot W_Q = 64 \times 128 \times 128 \times 32 = 64 \times 32$

Attn weigh score = $Q \cdot K^T$ = 64 × 32 × 32 × 64

Sx = 64 × 64

7) What will be shape of the mask M?

mask Shape is same as of Attn Score

∴ mask shape = 64 × 64

- [ ] 16×16
- [ ] 32×32
- [x] 64×64
- [ ] 128×128
- [ ] None of these.

8) Consider the following set of GPT-based text generative models (represented by M_i with $i \in \{1, 2, 3, 4\}$) and the corresponding text generated with the prompt "I like"

M_1 : I like to to like.
M_2 : I like like like.
M_3 : I like the do because.
M_4 : I like to study the origin of the universe.

Select the model(s) that is(are) degenerative.

- [x] M_1
- [x] M_2
- [x] M_3
- [ ] M_4

A **degenerative** model in the context of text generation is one that produces repetitive, nonsensical, or low-quality text that lacks meaningful progression. Let's analyze the outputs from each model:

∴ M1 — | degenerative
M2 — |
M3 — |